

Automatización de evaluaciones de seguridad web mediante un Sistema WASA Automatizado Multi-herramienta y Visualización Dinámica de Vulnerabilidades Orquestado con n8n.

Facultad de Ingeniería

Carrera: Tecnicatura Universitaria en Programación

Autores: Lautaro Ferreria, Nicolás Navarrete

Tutores de tesis: Ariel Enferrel y Alberto Cortez

Línea de Investigación: Seguridad en Aplicaciones Web / DevSecOps

Mendoza – Argentina

2026

Resumen

Esta investigación presenta el diseño y desarrollo de una plataforma avanzada para la automatización de procesos de **Web Application Security Assessment (WASA)**. A diferencia de las auditorías manuales tradicionales, que resultan costosas en tiempo y propensas a inconsistencias, este sistema propone un modelo de **Seguridad como Código (SaC)** capaz de ejecutar análisis continuos y repetibles.

La solución integra herramientas líderes en la industria como **OWASP ZAP**, **ffuf**, **SQLMap** y **Nuclei**. La innovación principal radica en una arquitectura de microservicios orquestada mediante **n8n**, la cual utiliza **Redis (Memurai)** como *Message Broker* para desacoplar las tareas de escaneo pesado. Esta implementación permite que los análisis de alta intensidad (como inyecciones SQL profundas) sean procesados de forma asíncrona por un **Worker independiente en Python**, eliminando bloqueos en el flujo principal y optimizando la escalabilidad sobre múltiples activos. Los hallazgos se centralizan en una base de datos **PostgreSQL**, alimentando un Dashboard interactivo en **React** con backend en **Node.js**. Este ecosistema permite visualizar métricas críticas y la evolución histórica de las vulnerabilidades, transformando datos técnicos en información estratégica para la toma de decisiones.

Palabras clave: Fuzzing, n8n, Redis, Automatización, SQLMap, Seguridad Web, DevSecOps, Asincronía.

Abstract

This research presents the design and development of an advanced platform for automating **Web Application Security Assessment (WASA)** processes. Unlike traditional manual audits, which are time-consuming and prone to inconsistencies, this system proposes a **Security as Code (SaC)** model capable of performing continuous and repeatable analyses.

The solution integrates industry-leading tools such as **OWASP ZAP**, **ffuf**, **SQLMap**, and **Nuclei**. The core innovation lies in a microservices architecture orchestrated by **n8n**, utilizing **Redis (Memurai)** as a message broker to decouple heavy scanning tasks. This implementation allows high-intensity analyses (such as deep SQL injections) to be processed asynchronously by an **independent Python Worker**, eliminating bottlenecks in the primary workflow and optimizing scalability across multiple assets. Findings are centralized in a **PostgreSQL** relational database, feeding an interactive dashboard developed in **React** with a **Node.js** backend. This ecosystem enables the visualization of critical metrics and the historical evolution of vulnerabilities, transforming complex technical data into strategic information for decision-making.

Keywords: Fuzzing, n8n, Redis, Automation, SQLMap, Web Security, DevSecOps, Asynchrony.

Índice

Resumen	2
1. INTRODUCCIÓN	7
1.1 Contexto del problema	7
1.2 Planteamiento del problema	7
1.3 Justificación	7
1.4 Objetivos	7
1.4.1 Proposición de diseño	8
1.5 Alcance y limitaciones	8
2. MARCO TEÓRICO	9
2.1 Seguridad en aplicaciones web	9
2.2 Amenazas y vulnerabilidades comunes	9
2.3 Modelos de seguridad y automatización (DevSecOps)	10
2.4 Herramientas de análisis y orquestación	11
2.4.1 Orquestación de flujos con n8n	11
2.4.2 Motores de escaneo (Scanning Engines)	12
2.4.3 Arquitectura de visualización y gestión de datos	12
2.4.4 Intermediación de mensajes y gestión de colas (Redis / Memurai)	12
2.5 Estado del arte: Sistemas automatizados de WASA y brechas identificadas	13
2.5.1 Estudios comparativos de herramientas DAST	13
2.5.2 Integración de seguridad en pipelines CI/CD (DevSecOps)	13
2.5.3 Trabajos sobre visualización de métricas de seguridad	14
2.5.4 Brechas identificadas y posicionamiento de este trabajo	14
3. MARCO METODOLÓGICO	15
3.1 Tipo de investigación	15
3.2 Diseño de investigación	16
3.3 Dimensiones de análisis del estudio de caso	17
3.4 Técnicas de recolección de datos	18
3.5 Instrumentos	19
3.6 Criterios de validación del sistema	20
3.7 Parámetros del entorno experimental	22
3.7.1 Entorno objetivo (Target)	22
3.7.2 Entorno de orquestación y herramientas	23
3.7.3 Versiones de los motores de escaneo	23
3.7.4 Wordlists y plantillas utilizadas	23
3.8 Consideraciones éticas y de alcance	23
4. DESARROLLO DEL ESTUDIO	24
4.1 Arquitectura del Sistema Propuesto	24
4.2 Implementación y Configuración de Herramientas	26
4.2.1 Orquestador Dinámico (n8n)	26
4.2.2 Integración de Motores de Seguridad	26
4.2.3 Servidor de Visualización y Dashboard	26

4.2.4 Implementación del Sistema de Fuzzing Asíncrono	27
4.3 Procedimiento Experimental (Flujo de Ejecución)	27
4.3.1 Procedimiento de Ejecución con Desacoplamiento	28
5. RESULTADOS	29
5.1 Ejecución del escaneo automatizado	29
5.2 Hallazgos por motor y cobertura del escaneo	29
5.2.1 Resultados de Nuclei	30
5.2.2 Resultados de OWASP ZAP	31
5.2.3 Resultados de ffuf	32
5.2.4 Resultados de SQLMap (Worker asíncrono)	32
5.3 Distribución de severidad y clasificación taxonómica	33
5.4 Normalización y eliminación de duplicados	35
5.5 Ranking de endpoints vulnerables	35
5.6 Verificación de criterios de validación	36
6. ARQUITECTURA E INTERFAZ DE VISUALIZACIÓN	37
6.1 Introducción a la Generalidad de la Interfaz de Visualización	37
6.2 Diseño y Lógica del Funcionamiento Interno (Backend)	38
6.3 Estructura y Procesamiento de Datos (Frontend)	39
6.4 Interfaz de Usuario Externa (UI) y Visualización	39
6.4.1 Filtros Dinámicos e Indicadores Clave (KPIs)	39
6.4.2 Visualización de Datos mediante Gráficos (Recharts)	40
6.4.3 Reporte Detallado y Componente Modal	41
6.4.4 Ranking de Endpoints Vulnerables	43
7. DISCUSIÓN	44
7.1 Automatización orquestada frente al análisis manual: qué queda demostrado y qué no	44
7.2 Sinergia multi-motor: evidencia de máxima amplitud	45
7.3 La arquitectura asíncrona con Redis: validación funcional, hallazgo de seguridad y alcance de la evidencia	46
7.4 Normalización y deduplicación: la capa de datos como contribución técnica	47
7.4.1 Corrección del mapeo de severidad y verificación posterior	47
7.4.2 Segundo ciclo de verificación y evidencia de reproducibilidad	48
7.5 El Dashboard como herramienta de gestión de riesgo: valor presente y extensiones	49
7.6 Tensiones metodológicas del diseño y amenazas a la validez	49
7.7 Posicionamiento de la contribución en el campo y grado de verificación	51
7.8 Falsos positivos: límite estructural de la validación y agenda pendiente	52
8. CONCLUSIONES	53
8.1 Viabilidad del Monitoreo Continuo (Dev SecOps)	53
8.2 Sinergia y Eficiencia Multi-Motor	53
8.3 Solución a la Fragmentación de Datos	54
8.4 Transformación de Datos en Inteligencia Accionable	54
8.5 Líneas de Trabajo Futuro	54
LT1 — Integración nativa del sistema en pipelines de CI/CD con activación por evento de commit	54

LT2 — Incorporación de análisis SAST como capa complementaria al pipeline DAST	55
LT3 — Escalabilidad horizontal del Worker asíncrono mediante un pool de consumidores Redis	56
LT4 — Módulo de priorización de remediación basado en scoring contextual y deuda técnica acumulada	57
LT5 — Validación del sistema sobre aplicaciones reales en entornos de staging con autorización explícita	58
LT6 — Incorporación de un módulo de aprendizaje automático para reducción adaptativa de falsos positivos	59
LT7 — Validación exhaustiva del pipeline de normalización	60
Reflexión de cierre	60
9. REFERENCIAS BIBLIOGRÁFICAS	61
10. ANEXOS	63

Índice de tablas

Tabla 1. Fases del diseño de investigación y sus resultados esperados	17
Tabla 2. Dimensiones de análisis del estudio del caso, organizados por tipo	17
Tabla 3. Instrumentos tecnológicos del estudio del caso, organizados por capa lógica	19
Tabla 4. Criterios de validación del sistema con sus indicaciones y métodos de verificación	21
Tabla 5. Resumen de hallazgos por motor — scan 24/05/2026	30
Tabla 6. Distribución de hallazgos por severidad y motor — scan 24/05/2026	33
Tabla 7. Clasificación de vulnerabilidades por CWE y categoría OWASP Top 10:2021 — scan 24/05/2026	33
Tabla 8. Ranking de endpoints por concentración de vulnerabilidades — scan 24/05/2026	36
Tabla 9. Verificación de criterios de validación del sistema — scan 24/05/2026	37
Tabla 10. Comparación de ciclos de verificación post-corrección (evidencia de reproducibilidad)	48
Tabla 11. Tensiones metodológicas del experimento y sus implicancias para la validez del sistema	50

Tabla 12. Contribuciones del trabajo y grado de verificación empírica — scan 24/05/2026

51

Índice de figuras

Figura 1. Arquitectura general del sistema propuesto y separación en cuatro capas lógicas	25
Figura 2. Flujo de comunicación asíncrona entre n8n, Redis (Memurai) y el Worker de SQLMap	28
Figura 3. Lógica de construcción dinámica de consultas SQL en el backend	38
Figura 4. Petición asíncrona desde el frontend al servidor evidenciando el paso de parámetros por URL	39
Figura 5. Cabecera del Dashboard mostrando filtros de trazabilidad y tarjetas KPI	40
Figura 6. Representación gráfica de severidades y evolución temporal de escaneos	41
Figura 7. Interfaz modal exhibiendo los datos generales de cada una de las vulnerabilidades detectadas	41
Figura 8. Interfaz modal exhibiendo los metadatos, remediación y evidencia técnica de una vulnerabilidad específica	43
Figura 9. Vista de priorización mostrando el ranking de endpoints según la concentración de vulnerabilidades detectadas	44
Figura G.1. Cabecera del Dashboard — scan_id=105 (03/08/2026, 00:11 hs)	75
Figura G.2. Cabecera del Dashboard — scan_id=108 (03/08/2026, 10:33 hs)	75

1. INTRODUCCIÓN

1.1 Contexto del problema

En el panorama actual de la ciberseguridad, las aplicaciones web constituyen la principal superficie de ataque para las organizaciones. El proceso estándar para evaluar su resistencia se denomina *Web Application Security Assessment* (WASA). Este proceso implica una revisión exhaustiva de la aplicación para identificar fallos de seguridad antes de que sean explotados.

Tradicionalmente, el WASA se ha realizado de forma manual o mediante escaneos aislados. Sin embargo, en entornos de desarrollo ágil, la velocidad de despliegue supera la capacidad de los equipos de seguridad para realizar pruebas manuales. Los análisis manuales consumen una cantidad excesiva de horas-hombre, son difíciles de escalar a múltiples dominios y carecen de la consistencia necesaria para garantizar que cada cambio en el código sea evaluado bajo los mismos estándares.

1.2 Planteamiento del problema

La falta de automatización en las evaluaciones de seguridad genera "ventanas de exposición" donde las vulnerabilidades permanecen activas durante semanas. Además, el uso de múltiples herramientas de seguridad independientes suele fragmentar la información, dificultando la correlación de datos y la visualización de riesgos en tiempo real.

A pesar de existir herramientas de escaneo potentes, su ejecución suele ser reactiva y manual. La pregunta de investigación que guía este trabajo es: **¿Cómo puede optimizarse la cobertura y consistencia de un Web Application Security Assessment mediante un sistema orquestado que permita el escaneo continuo y la centralización de métricas en un dashboard dinámico?**

1.3 Justificación

La implementación de este sistema se justifica por la necesidad de transitar hacia un modelo de seguridad proactiva. Al automatizar el ciclo de vida del descubrimiento y fuzzing, se mejora significativamente la **cobertura** (al analizar más endpoints en menos tiempo), la **repetibilidad** (asegurando que los mismos tests se apliquen siempre) y la **consistencia** del análisis. La inclusión de un panel de visualización resuelve la brecha entre la obtención de datos técnicos y el análisis de gestión, permitiendo identificar rápidamente los endpoints más vulnerables y la efectividad de las herramientas utilizadas.

1.4 Objetivos

Objetivo General

Diseñar e implementar una plataforma automatizada para la ejecución continua de *Web Application Security Assessments* (WASA), orquestada mediante n8n, que integre herramientas de análisis de seguridad (OWASP ZAP, ffuf, SQLMap, Nuclei) y centralice la información en un *dashboard* interactivo, con el fin de diseñar y validar la cobertura, consistencia y visualización de vulnerabilidades en aplicaciones web.

Objetivos Específicos

1. **Orquestar el flujo de evaluación continua:** Configurar flujos de trabajo en n8n que incluyan disparadores de tiempo (*Schedule*) e iteraciones dinámicas (*Loop*) para automatizar escaneos periódicos sobre múltiples objetivos.
2. **Integrar motores de análisis avanzados:** Acoplar herramientas de seguridad comprobadas, destacando la implementación de **Nuclei** para la detección rápida de exposiciones y CVEs mediante plantillas YAML, junto a ZAP, ffuf y SQLMap.
3. **Desarrollar una arquitectura de persistencia y distribución:** Crear una base de datos relacional (PostgreSQL) para el almacenamiento estructurado y desarrollar una API RESTful intermedia (con Node.js y Express) que normalice y sirva los datos recolectados.
4. **Implementar visualización de métricas (Dashboard):** Desarrollar una interfaz gráfica de usuario utilizando React para presentar de manera ejecutiva los resultados del escaneo, mostrando la distribución de vulnerabilidades por severidad, su evolución histórica y los *endpoints* de mayor riesgo.

1.4.1 Proposición de diseño

En el marco del paradigma de *Design Science Research* (Hevner et al., 2004; Wieringa, 2014), el presente trabajo se estructura en torno a la siguiente proposición de diseño:

Una plataforma de Web Application Security Assessment que orqueste de forma continua y automatizada múltiples motores de análisis especializados (OWASP ZAP, Nuclei, ffuf y SQLMap) mediante un flujo de trabajo n8n con desacoplamiento asíncrono basado en Redis, y que centralice los hallazgos normalizados bajo los estándares OWASP Top 10:2021 y CWE en un dashboard interactivo, logrará una cobertura de detección cualitativamente superior —en variedad de categorías CWE cubiertas— a la obtenible por cualquiera de dichas herramientas operando de forma aislada, y lo hará de manera autónoma y repetible sobre el mismo conjunto de activos.

Esta proposición se apoya en la evidencia empírica que muestra que ninguna herramienta DAST individual cubre el espectro completo de vulnerabilidades del OWASP Top 10 (Somi, 2024; Qadir et al., 2025), y en la brecha documentada entre la disponibilidad de herramientas y su integración continua en pipelines reales (Cheenepalli et al., 2025). La validez de la proposición **P1** se verifica a través de los criterios de validación definidos en la sección 3.6 y los resultados reportados en el capítulo 5, en particular mediante la Tabla 7, que demuestra que al menos dos motores aportan hallazgos en categorías CWE que los demás no cubren.

1.5 Alcance y limitaciones

Alcance El proyecto abarca el diseño de la arquitectura completa (Frontend, Backend, Base de Datos y Orquestador) y la configuración de las integraciones necesarias para que el sistema funcione de manera autónoma.

- El sistema será capaz de escanear aplicaciones web predefinidas (utilizando entornos controlados como *Damn Vulnerable Web Application* - DVWA para las pruebas de validación).

- Se contempla la normalización de datos básicos de vulnerabilidades (categorización OWASP y CWE ID) extraídos de los reportes generados por las herramientas subyacentes. La asignación de un puntaje CVSS normalizado por hallazgo no forma parte del alcance de la versión actual del pipeline y se propone como línea de trabajo futuro (ver LT4, sección 8.5).
- El *dashboard* proporcionará visualización en tiempo real de los datos almacenados en la base de datos tras cada ciclo de escaneo programado.

Limitaciones

- **Remediación:** La plataforma está diseñada exclusivamente para la *detección y visualización* de vulnerabilidades (WASA). No incluye capacidades de parcheo automático ni remediación activa (Auto-remediation) de los fallos encontrados.
- **Falsos positivos/negativos:** Al depender de motores de escaneo de terceros (ZAP, Nuclei, etc.), el sistema hereda la tasa de precisión de estas herramientas, requiriendo en última instancia la validación manual por parte de un analista de seguridad para confirmar hallazgos complejos. El tratamiento analítico de esta limitación en el contexto del diseño experimental adoptado se desarrolla en la sección 7.8.
- **Impacto en el objetivo:** Debido a la naturaleza agresiva de técnicas como el *fuzzing* o la inyección SQL automatizada, el alcance de las pruebas está limitado a entornos de desarrollo, *staging* o laboratorios controlados, ya que su ejecución en entornos de producción sin las precauciones debidas podría causar denegación de servicio (DoS) o corrupción de datos.

2. MARCO TEÓRICO

2.1 Seguridad en aplicaciones web

La seguridad informática en el ámbito web se centra en proteger la tríada de la información: Confidencialidad, Integridad y Disponibilidad (CID). Las aplicaciones web constituyen hoy la principal superficie de ataque para las organizaciones: según el Informe de Investigaciones de Brechas de Datos de Verizon (DBIR 2023), los ataques dirigidos a aplicaciones web representan cerca del 40% de la totalidad de brechas de seguridad reportadas, con un costo promedio por incidente que supera los 4,45 millones de dólares según el estudio de IBM Cost of a Data Breach Report del mismo año (Somi, 2023).

En este contexto, el Web Application Security Assessment (WASA) se define como el proceso sistemático de identificar, analizar y reportar vulnerabilidades en una aplicación web, simulando las tácticas de un adversario real para fortalecer las defensas antes de que sean explotadas. A diferencia de las pruebas de penetración puntuales, un WASA continuo —objetivo central de este trabajo— busca institucionalizar la evaluación de seguridad como parte del ciclo de vida del desarrollo, en línea con los principios del paradigma DevSecOps (Cheenepalli et al., 2025).

2.2 Amenazas y vulnerabilidades comunes

Para estandarizar el análisis de riesgos, esta investigación se apoya en el OWASP Top 10 (2021), el documento de referencia mundial que categoriza los riesgos más críticos en aplicaciones web. Su relevancia académica está ampliamente documentada: estudios empíricos recientes sobre herramientas de seguridad utilizan este estándar como marco de clasificación (Qadir et al., 2025; Priyawati et al., 2022). Las categorías de mayor relevancia para el presente trabajo son:

- A01:2021 — Control de Acceso Roto (Broken Access Control / CWE-22): Incluye vulnerabilidades como el Directory Traversal, donde un atacante puede leer archivos arbitrarios del servidor manipulando rutas de archivo.
- A03:2021 — Inyección (Injection / CWE-89): Fallos donde datos no confiables son interpretados como comandos. La Inyección SQL (SQLi) es el caso más prevalente y de mayor severidad (CVSS típicamente entre 7.5 y 9.8).
- A07:2021 — Fallos de Identificación y Autenticación: Incluye la ausencia de tokens Anti-CSRF (CWE-352), que permite a un atacante forzar acciones en nombre de un usuario autenticado.
- Exposición de datos sensibles y configuraciones incorrectas: Detectables mediante herramientas de análisis basadas en plantillas como Nuclei, que compara el comportamiento del servidor contra bases de datos de CVEs conocidos (ProjectDiscovery, 2025).

El complemento al OWASP Top 10 es el Common Weakness Enumeration (CWE), mantenido por el consorcio MITRE (MITRE, 2025), que provee identificadores únicos y estandarizados para cada tipo de debilidad de software. El uso combinado de OWASP Top 10 y CWE como sistema de clasificación dual es una práctica recomendada en la literatura reciente: el estudio de Somi (2024) en el Journal of Engineering and Applied Sciences Technology es el primero en mapear empíricamente los hallazgos de herramientas DAST simultáneamente contra ambos estándares en una muestra de aplicaciones reales.

2.3 Modelos de seguridad y automatización (DevSecOps)

El paradigma tradicional de seguridad al final del ciclo ha evolucionado hacia el DevSecOps, metodología que integra prácticas de seguridad en cada etapa del ciclo de vida del desarrollo de software (SDLC). Una revisión multivocal de la literatura (Multivocal Literature Review) publicada en el International Journal of Information Security identificó 39 prácticas DevSecOps distintas a partir de 103 estudios primarios, constatando además un crecimiento sostenido en el número de publicaciones sobre el tema entre 2018 y 2023 (Prates & Pereira, 2024).

Dentro de este paradigma, los conceptos fundamentales para el sistema desarrollado en esta tesis son:

- Security as Code (SaC): Metodología que codifica las políticas y pruebas de seguridad de forma que puedan ejecutarse automáticamente como parte del pipeline de CI/CD. Putra y Kabetta (2022) demostraron en la Conferencia IEEE ICOSNIKOM que la integración simultánea de pruebas SAST y DAST en

pipelines de CI/CD produce tasas de detección de vulnerabilidades significativamente superiores a las metodologías de prueba aisladas.

- **DAST (Dynamic Application Security Testing):** Técnica de análisis de caja negra que examina la aplicación en tiempo de ejecución, sin acceso al código fuente. Constituye el enfoque metodológico central de este trabajo. Su ventaja fundamental es la capacidad de detectar vulnerabilidades que solo se manifiestan en tiempo de ejecución, como fallos de inyección SQL o problemas de configuración de sesión (Somi, 2023).
- **Fuzzing:** Técnica que consiste en generar y enviar grandes volúmenes de datos —parcialmente malformados o estructurados según payloads conocidos— a las entradas de una aplicación para forzar comportamientos inesperados que revelen vulnerabilidades. Su efectividad ha sido demostrada en múltiples estudios: Klooster et al. (2023), en el taller SBFT'23 de IEEE/ACM, demostraron que incluso sesiones de fuzzing de 15 minutos integradas en pipelines de CI son capaces de descubrir errores críticos.

A los fines prácticos de esta tesis, el término se emplea en sentido amplio, abarcando la ejecución automatizada y orquestada de múltiples herramientas DAST (ffuf, SQLMap, Nuclei, ZAP) más allá del fuzzing de payloads en sentido estricto.

Un aspecto crítico de la automatización de pruebas de seguridad que esta tesis aborda directamente es el problema de la "fatiga de alertas": la literatura documentada por Cheenepalli et al. (2025) indica que, a pesar de que el 68% de las organizaciones han implementado algún nivel de DevSecOps, solo el 12% realiza análisis de seguridad por commit, evidenciando que la barrera no es la disponibilidad de herramientas sino la dificultad para integrarlas y priorizar sus resultados de forma automatizada y continua.

2.4 Herramientas de análisis y orquestación

En este apartado se describen las tecnologías que conforman el núcleo del sistema propuesto, con referencia a los estudios que respaldan su selección.

2.4.1 Orquestación de flujos con n8n

n8n es una plataforma de automatización de flujos de trabajo de código abierto basada en nodos que permite conectar diversas aplicaciones y servicios mediante una interfaz visual. A diferencia de soluciones de orquestación de pipelines de seguridad como Jenkins o GitLab CI —cuyo uso requiere conocimiento de lenguajes de configuración específicos (YAML/Groovy)— n8n permite diseñar la lógica del escaneo de forma visual, reduciendo la barrera de adopción. En el contexto de este trabajo, sus nodos de control más relevantes son:

- **Schedule Trigger:** Permite la ejecución periódica y programada de tareas, implementando el principio de monitoreo continuo.
- **Loop Over Items:** Permite la iteración dinámica sobre una lista de activos o URLs descubiertas, garantizando la escalabilidad del análisis a múltiples objetivos en una sola ejecución.
- **Execute Command:** Permite invocar los binarios del sistema operativo (CLI de ffuf, Nuclei, SQLMap) inyectando variables del objetivo en cada iteración.

2.4.2 Motores de escaneo (Scanning Engines)

La selección de las herramientas de escaneo se fundamenta en la literatura de comparación empírica de herramientas DAST. Albahar, Alansari & Jurcut (2022) y Altulaihan, Alismail & Frikha (2023) identificaron a Burp Suite y OWASP ZAP como las herramientas DAST de código abierto con mayor presencia en la literatura académica. Aydos et al. (2022), en un estudio de mapeo sistemático de 80 artículos sobre pruebas de seguridad en aplicaciones web, encontraron que ZAP es la herramienta más citada, presente en 48 de los 80 trabajos analizados. Sobre esta base, las herramientas integradas en el sistema son:

- OWASP ZAP: Escáner dinámico que combina descubrimiento de rutas (Spidering) y análisis activo de vulnerabilidades. Un estudio empírico sobre la efectividad de ZAP para detectar vulnerabilidades del OWASP Top 10 en 70 aplicaciones web distintas confirmó su cobertura en categorías de inyección, autenticación rota y exposición de datos (Khanum et al., 2023, como se cita en Qadir et al., 2025).
- Nuclei (ProjectDiscovery): Herramienta de escaneo basada en plantillas YAML actualizadas por la comunidad de seguridad que permite detectar CVEs y configuraciones incorrectas de forma extremadamente rápida. Somi (2024) la incluye en su benchmarking comparativo de herramientas DAST como referencia de eficiencia en la detección de vulnerabilidades conocidas con bajo consumo de recursos.
- ffuf: Fuzzer de alta velocidad para descubrimiento de directorios y archivos ocultos mediante fuerza bruta sobre wordlists. Complementa a ZAP en la fase de reconocimiento, ampliando el mapa de superficie de ataque.
- SQLMap: Herramienta especializada en la detección y explotación automática de inyecciones SQL. Su ejecución en modo no supervisado (batch) requiere tiempos prolongados de análisis, lo que motivó el diseño de la arquitectura asíncrona con Redis descrita en la sección 4.2.4.

2.4.3 Arquitectura de visualización y gestión de datos

Para transformar los hallazgos técnicos en información accionable, el sistema implementa una arquitectura de software de tres capas:

- PostgreSQL: Sistema de gestión de bases de datos relacional donde se estructuran los hallazgos históricos. El diseño del esquema normalizado permite consultas analíticas temporales, habilitando la comparación entre ciclos de escaneo.
- Backend Node.js y Express: API RESTful que actúa como intermediario entre la base de datos y la interfaz de usuario. Implementa filtrado del lado del servidor (Server-Side Filtering) para optimizar el manejo de conjuntos de datos de gran volumen.
- Frontend React: Biblioteca de JavaScript para la construcción de la interfaz gráfica de usuario (SPA). Utiliza la biblioteca Recharts para la visualización de métricas mediante gráficos SVG responsivos.

2.4.4 Intermediación de mensajes y gestión de colas (Redis / Memurai)

Para resolver la problemática de los tiempos de ejecución prolongados característicos de las herramientas de análisis profundo como SQLMap, se incorpora un sistema de mensajería asíncrona basado en el patrón Productor-Consumidor:

- Redis (Remote Dictionary Server): Actúa como Message Broker de baja latencia que permite el desacoplamiento entre el orquestador (n8n) y los motores de ejecución de alta intensidad, eliminando bloqueos en el flujo principal.
- Memurai: Implementación nativa de Redis para entornos Windows que garantiza compatibilidad total con el protocolo de red Redis sin la sobrecarga de capas de virtualización adicionales.

2.5 Estado del arte: Sistemas automatizados de WASA y brechas identificadas

Esta sección revisa los trabajos académicos previos más relevantes en el área de automatización de evaluaciones de seguridad en aplicaciones web, con el objetivo de posicionar la propuesta de este trabajo en relación con el conocimiento existente e identificar las brechas que justifican su desarrollo.

2.5.1 Estudios comparativos de herramientas DAST

La investigación empírica sobre herramientas DAST ha experimentado un crecimiento notable. En su estudio de mapeo sistemático de literatura, Aydos et al. (2022) analizaron 80 artículos sobre pruebas de seguridad en aplicaciones web, constatando que el 61% de los trabajos estudiados se concentra en métodos y técnicas para el testing de seguridad, mientras que sólo 7 de 80 introducen frameworks para mejorar el proceso de evaluación. Esta desproporción evidencia una brecha entre la investigación centrada en herramientas individuales y la ingeniería de sistemas que integre múltiples herramientas de forma orquestada.

El trabajo de Somi (2024) en el Journal of Engineering and Applied Sciences Technology realizó un benchmarking comparativo de las principales soluciones DAST disponibles, incluyendo OWASP ZAP y Nuclei, evaluando su eficacia, eficiencia y tasa de falsos positivos. Sus hallazgos confirman que ninguna herramienta individual cubre el espectro completo de vulnerabilidades del OWASP Top 10, lo que fundamenta el enfoque multi-motor adoptado en esta tesis.

Más recientemente, el estudio de revisión publicado por Qadir et al. (2025) en PeerJ Computer Science es el primero en evaluar empíricamente 4 herramientas SAST y 5 DAST sobre 75 aplicaciones web reales, mapeando los hallazgos tanto contra el OWASP Top 10:2021 como contra el CWE Top 25:2023. Sus conclusiones subrayan que la adopción conjunta de SAST y DAST es ampliamente recomendada en la literatura pero raramente evaluada de forma comparativa y sistemática, dejando un vacío en la evidencia empírica sobre qué combinación de herramientas ofrece mayor cobertura.

2.5.2 Integración de seguridad en pipelines CI/CD (DevSecOps)

La integración de pruebas de seguridad automatizadas en pipelines de entrega continua ha sido objeto de creciente atención académica. Putra y Kabetta (2022), en la Conferencia IEEE ICOSNIKOM, presentaron una implementación práctica de DevSecOps que integra pruebas SAST y DAST en pipelines de CI/CD, demostrando mejoras cuantificables en la

tasa de detección de vulnerabilidades. Singh (2020) en SSRN ofrece un estudio comprensivo de herramientas de automatización de pruebas de seguridad en pipelines, comparando soluciones de código abierto y comerciales, con particular énfasis en las implicaciones sobre la velocidad de entrega.

Abiola y Olufemi (2023) propusieron en el International Journal of Computer Applications un pipeline de CI/CD mejorado bajo el enfoque DevSecOps, destacando que los equipos que automatizan los controles de seguridad reducen significativamente la "ventana de exposición" —el período entre la introducción de una vulnerabilidad en el código y su detección— problema que esta tesis aborda directamente mediante la ejecución periódica orquestada con n8n.

En el ámbito del fuzzing continuo integrado en CI/CD, Klooster et al. (2023), en el Taller SBFT'23 del IEEE/ACM International Conference on Automated Software Engineering, demostraron que el fuzzing en sesiones cortas y frecuentes dentro de pipelines de CI es efectivo y escalable, sentando una base empírica para el modelo de "fuzzing continuo" adoptado en este trabajo. Nourry et al. (2023), en ACM Transactions on Software Engineering and Methodology, complementaron estos hallazgos analizando los desafíos prácticos que enfrentan los equipos de desarrollo durante las actividades de fuzzing, incluyendo la dificultad de interpretar volúmenes masivos de alertas —problema que el Dashboard interactivo desarrollado en esta tesis busca mitigar.

2.5.3 Trabajos sobre visualización de métricas de seguridad

La visualización de resultados de seguridad como componente del proceso de evaluación ha recibido menor atención académica que las propias herramientas de detección. Sin embargo, la literatura sobre DevSecOps identifica consistentemente la "fatiga de alertas" como uno de los principales obstáculos para la adopción efectiva de herramientas de análisis automatizado (Cheenepalli et al., 2025). Los sistemas de gestión de vulnerabilidades como DefectDojo (de código abierto) o Faraday abordan parcialmente este problema, pero están orientados a flujos de trabajo de equipos de seguridad profesionales, no a la automatización de escaneos periódicos sobre activos específicos con visualización histórica integrada.

2.5.4 Brechas identificadas y posicionamiento de este trabajo

Del análisis de la literatura revisada se identifican las siguientes brechas que justifican el desarrollo de la plataforma propuesta en esta tesis:

- Ausencia de sistemas integrados de orquestación multi-herramienta: La mayoría de los estudios evalúan herramientas DAST de forma individual o comparativa, pero no proponen ni implementan sistemas que las coordinen de forma automática y continua. El uso de n8n como orquestador para este tipo de pipelines de seguridad representa una contribución práctica no documentada en la literatura revisada.
- Brecha entre detección y visualización para no-especialistas: Los trabajos existentes se centran en la precisión de detección pero no abordan el problema de transformar el volumen de alertas en información de gestión accionable. El componente de Dashboard desarrollado en esta tesis responde directamente a la

necesidad documentada de reducir la fatiga de alertas (Cheenepalli et al., 2025; Nourry et al., 2023).

- Limitaciones del enfoque síncrono para herramientas de alta intensidad: La literatura existente no documenta soluciones específicas para el problema de bloqueo que genera la integración de herramientas como SQLMap en pipelines de orquestación tipo n8n. El patrón Productor-Consumidor con Redis implementado en este trabajo constituye una solución de ingeniería original al problema.

3. MARCO METODOLÓGICO

3.1 Tipo de investigación

El presente trabajo se enmarca dentro de una investigación de tipo aplicada y tecnológica, orientada a resolver un problema práctico del ámbito de la ciberseguridad: la ineficiencia operativa y la falta de continuidad en las auditorías de seguridad web en entornos de desarrollo ágil.

En cuanto al alcance, la investigación es de tipo descriptiva e instrumental. Es descriptiva porque sistematiza y caracteriza el comportamiento de las vulnerabilidades detectadas bajo el estándar OWASP Top 10 y la taxonomía CWE. Es instrumental porque el objetivo central del trabajo es el diseño, implementación y validación de un artefacto tecnológico —la plataforma WASA automatizada— cuya utilidad se evalúa en términos de su funcionamiento correcto y cobertura de detección sobre un entorno controlado.

En consecuencia, el diseño adoptado es el de un Estudio de Caso de Ingeniería de Software (Engineering Case Study), según la taxonomía de Wohlin et al. (2012), ampliamente utilizada en investigación empírica en ingeniería de software. Esta modalidad es la más adecuada cuando el objetivo es comprender en profundidad el comportamiento de un sistema en su contexto real de uso, con foco en la validación de que el sistema cumple los requisitos funcionales para los que fue diseñado, más que en la generalización estadística de los resultados.

Justificación del encuadre metodológico: La naturaleza del objeto de estudio —un sistema software integrado que orquesta múltiples herramientas de seguridad— determina la elección metodológica. Runeson y Höst (2009) establecen que el estudio de caso es el método preferido cuando se investiga un fenómeno contemporáneo en su contexto real, cuando el investigador tiene escaso control sobre los eventos, y cuando las preguntas de investigación son de tipo "cómo" o "por qué". Las tres condiciones se cumplen en este trabajo: el sistema se evalúa en un entorno de laboratorio que replica las condiciones de uso real, no se manipulan variables en el sentido clásico del experimento, y la pregunta de investigación indaga sobre el modo en que la orquestación automatizada mejora la cobertura y consistencia de los análisis de seguridad.

Articulación entre Design Science Research y el Estudio de Caso. El presente trabajo adopta el paradigma de *Design Science Research* (DSR) (Hevner et al., 2004; Wieringa, 2014) como marco general de la investigación: el objeto central del trabajo es el diseño, construcción y evaluación de un artefacto tecnológico —la plataforma WASA

automatizada— cuya contribución reside en la solución práctica que provee a un problema identificado en el campo del DevSecOps. Dentro de ese paradigma, el método empleado para *evaluar* el artefacto construido es el Estudio de Caso de Ingeniería de Software (Wohlin et al., 2012; Runeson & Höst, 2009), que opera como el mecanismo de validación instrumental descrito en los párrafos precedentes. Ambos marcos son compatibles y complementarios: DSR define *qué* se construye y *por qué* constituye una contribución, mientras que el Estudio de Caso define *cómo* se verifica que el artefacto cumple los requisitos para los que fue diseñado. Esta distinción es coherente con la taxonomía de Wieringa (2014), quien establece que en investigación de diseño el método de evaluación del artefacto puede ser independiente del paradigma que rige su construcción.

Uso de DVWA como entorno controlado: La selección de Damn Vulnerable Web Application (DVWA) como objetivo de pruebas no responde a una limitación del diseño, sino a una práctica metodológica estándar en la literatura de evaluación de herramientas de seguridad. Khanum et al. (2023, como se cita en Qadir et al., 2025) la utilizan en su estudio sobre 70 aplicaciones web; Albahar, Alansari & Jurcut (2022) la incluyen en su benchmarking empírico de herramientas DAST; y el estudio de Qadir et al. (2025) la adopta como uno de los entornos de referencia al evaluar 4 herramientas SAST y 5 DAST sobre aplicaciones reales. El uso de un entorno con vulnerabilidades conocidas e intencionadas es el método correcto para validar que el sistema de detección funciona según lo especificado: si el sistema no detectara las vulnerabilidades que DVWA garantiza, el artefacto quedaría refutado. La configuración en nivel 'Low' maximiza la superficie de ataque disponible, asegurando que ninguna vulnerabilidad quede fuera del alcance por restricciones del entorno objetivo, lo cual es el diseño más exigente para la validación de cobertura.

3.2 Diseño de investigación

El diseño se estructura como un estudio de caso único, de carácter holístico, desarrollado en un entorno de laboratorio aislado. La unidad de análisis es el sistema integrado de WASA automatizado (n8n + motores de escaneo + backend + dashboard) actuando sobre la aplicación objetivo DVWA. No se realizan pruebas sobre aplicaciones en producción ni de terceros, en cumplimiento de los principios éticos y legales que rigen las pruebas de seguridad ofensiva.

Sobre la continuidad del monitoreo: Una distinción conceptual fundamental de este trabajo es la diferencia entre el diseño del sistema para la ejecución continua y el ciclo de escaneo documentado con fines de validación formal. El sistema fue desarrollado y ejecutado de manera iterativa a lo largo del proceso de construcción, acumulando múltiples ciclos de escaneo cuya evidencia histórica queda registrada en la base de datos PostgreSQL y es visible en el componente de evolución temporal del Dashboard (sección 6). Para la validación formal reportada en el Capítulo 5, se seleccionó un ciclo de referencia ejecutado el 24 de mayo de 2026, cuyas condiciones están completamente documentadas (sección 3.7), garantizando la reproducibilidad del experimento. Esta práctica es coherente con el principio metodológico de Runeson y Höst (2009) de aislar un ciclo representativo para el análisis empírico dentro de un sistema de ejecución continua.

El diseño se estructura en cuatro fases operativas secuenciales:

Tabla 1. Fases del diseño de investigación y sus resultados esperados.

Fase	Actividad principal	Resultado esperado	Sección relacionada
1	Despliegue del entorno controlado (DVWA + Docker) y montaje de la arquitectura del sistema	Entorno objetivo funcional; herramientas de escaneo operativas y comunicadas con n8n y PostgreSQL	§ 3.4 A y B
2	Ejecución automatizada de ciclos de WASA mediante el workflow n8n (Schedule + Loop)	Logs de ejecución por ciclo; alertas en formato JSON emitidas por cada herramienta	§ 3.4 B
3	Normalización y consolidación de alertas en PostgreSQL mediante la API Node.js/Express	Tabla vulnerabilidades poblada; duplicados eliminados; taxonomía OWASP/CWE asignada	§ 3.4 C
4	Análisis de los hallazgos mediante el Dashboard React y consultas directas a PostgreSQL	Métricas de cobertura, distribución de severidad, contribución por herramienta y evolución histórica	§ 3.4 D

Cada fase genera artefactos verificables (logs, registros en base de datos, capturas del dashboard) que sirven como evidencia del funcionamiento del sistema y como base para el análisis de los resultados presentados en el Capítulo 5.

3.3 Dimensiones de análisis del estudio de caso

En un estudio de caso de ingeniería de software, las dimensiones de análisis son las propiedades observables del artefacto construido cuya medición permite evaluar si el sistema cumple los objetivos para los que fue diseñado (Wohlin et al., 2012). A diferencia de los estudios experimentales clásicos, no se trata de variables manipuladas y controladas en un diseño factorial, sino de características del sistema cuya observación en condiciones definidas constituye evidencia de validez instrumental.

Se adopta esta terminología en lugar de "variables independientes/dependientes" por coherencia con el diseño de estudio de caso instrumental declarado: en este diseño, la configuración del entorno (DVWA + orquestador) es fija y actúa como condición de observación, no como variable manipulada. Las dimensiones evaluadas son:

Tabla 2. Dimensiones de análisis del estudio del caso, organizados por tipo.

Dimensión	Tipo	Definición operacional	Instrumento de medición
Cobertura del escaneo	Dimensión de resultado	Cantidad de endpoints únicos descubiertos y analizados por ciclo de escaneo	Conteo de URLs únicas en tabla vulnerabilidades por scan_id
Distribución de severidad	Dimensión de resultado	Proporción de hallazgos por nivel CVSS (Crítico / Alto / Medio / Bajo)	Agrupación COUNT(*) GROUP BY severity en PostgreSQL

Dimensión	Tipo	Definición operacional	Instrumento de medición
Tasa de deduplicación	Dimensión de proceso	Porcentaje de alertas duplicadas eliminadas por el proceso de normalización del backend	Comparación entre alertas crudas recibidas por n8n vs. registros efectivamente insertados en BD
Contribución por herramienta	Dimensión de resultado	Cantidad de vulnerabilidades únicas detectadas por cada motor de escaneo (ZAP, Nuclei, ffuf, SQLMap)	COUNT(*) GROUP BY source en tabla vulnerabilities
Autonomía de ejecución	Dimensión funcional	Capacidad del orquestador de completar el ciclo completo sin intervención humana entre el disparo del Schedule y la inserción en BD	Verificación de timestamps de inicio y fin del ciclo en logs de n8n
Desacoplamiento o asíncrono	Dimensión arquitectural	Verificación de que el Worker de SQLMap procesa su tarea sin bloquear el flujo principal del orquestador	Comparación de timestamps: cierre del ciclo n8n vs. INSERT diferido del Worker en BD
Conjunto de activos evaluados (Target)	Condición de observación	Aplicación DVWA en entorno Docker local con nivel de seguridad configurado en 'Low'	Configuración del entorno de laboratorio (Docker Compose)
Configuración del orquestador	Condición de observación	Parámetros del workflow n8n: Schedule, Loop, nodos de ejecución de cada herramienta	Archivo de configuración Flujo_Fuzzing_N8N.json (Anexo C)

Nota metodológica: Las condiciones de observación (Target y configuración del orquestador) permanecen fijas a lo largo del experimento por diseño deliberado, no por limitación. En un estudio de caso instrumental orientado a la validación de un artefacto, la reproducibilidad del entorno de evaluación es un requisito de rigor, no una restricción. El propósito del ciclo documentado es demostrar que el sistema detecta lo que debe detectar, bajo condiciones que cualquier evaluador pueda replicar. Ninguna dimensión de resultado depende de la generosidad del entorno: DVWA en nivel 'Low' garantiza la presencia de las vulnerabilidades objetivo, lo que convierte a cualquier fallo de detección en evidencia concluyente de deficiencia del sistema.

3.4 Técnicas de recolección de datos

La recolección de datos no se realiza mediante encuestas o entrevistas, sino a través de técnicas de observación automatizada y extracción de registros (logs y reportes) generados por el propio sistema durante su ejecución. Esta es la modalidad estándar en investigación empírica sobre sistemas software (Wohlin et al., 2012) y en trabajos de benchmarking de herramientas DAST (Somi, 2024; Qadir et al., 2025).

Las técnicas empleadas son:

- Ejecución de ataques controlados y automatizados: El orquestador n8n dispara los motores de escaneo sobre DVWA, capturando las respuestas HTTP, códigos de estado y payloads exitosos. La información cruda —emitida en formato JSON por Nuclei y ZAP, o como salida de texto estructurado por SQLMap y ffuf— es interceptada, procesada y transformada en un conjunto de datos relacional estandarizado.
- Consultas analíticas sobre la base de datos: Una vez concluido cada ciclo de escaneo, se ejecutan consultas SQL directas sobre la tabla vulnerabilities de PostgreSQL para obtener conteos por severidad, por herramienta (source) y por endpoint (url). Estas consultas son la fuente primaria de las métricas reportadas en el Capítulo 5.
- Observación directa del Dashboard interactivo: El Dashboard React actúa como instrumento de visualización y análisis, permitiendo verificar visualmente la coherencia entre los datos almacenados y su representación gráfica. Se documentan capturas de pantalla de cada vista lógica del dashboard como evidencia del funcionamiento del sistema.
- Verificación cruzada entre fuentes: Para garantizar la consistencia interna de los datos, los KPIs presentados en el Dashboard son contrastados contra los resultados de consultas SQL directas sobre la misma tabla y el mismo scan_id. Esta triangulación de fuentes —Dashboard, API REST y base de datos— constituye el mecanismo de validación de la integridad del pipeline de datos.

3.5 Instrumentos

Los instrumentos tecnológicos se organizan en cuatro capas lógicas. Para cada uno se detalla su rol específico en la recolección y procesamiento de datos:

Tabla 3. Instrumentos tecnológicos del estudio del caso, organizados por capa lógica.

Capa	Instrumento	Rol en la recolección de datos	Dato que produce
Objetivo	DVWA (Docker)	Entorno de pruebas con vulnerabilidades conocidas e intencionadas (SQLi, XSS, LFI). Provee la superficie de ataque controlada y reproducible.	Respuestas HTTP ante payloads de ataque
Orquestación	n8n	Coordina la ejecución secuencial y paralela de las herramientas. Captura sus salidas JSON y las persiste en la base de datos.	Logs de ejecución; timestamps de inicio/fin de cada ciclo
Escaneo	OWASP ZAP	Spidering de la aplicación objetivo y escaneo activo DAST. Emite alertas en JSON vía API REST local.	Alertas con tipo, CWE ID, URL, evidencia y severidad
Escaneo	Nuclei	Detección de CVEs y configuraciones incorrectas mediante plantillas YAML. Ejecución vía CLI.	Hallazgos con nombre de plantilla, severidad y URL afectada

Capa	Instrumento	Rol en la recolección de datos	Dato que produce
Escaneo	ffuf	Descubrimiento de directorios y archivos ocultos por fuerza bruta sobre wordlist.	Lista de rutas válidas (código HTTP 200/301/403)
Escaneo	SQLMap (Worker)	Detección y explotación automática de inyecciones SQL. Ejecutado de forma asíncrona mediante Worker Python + Redis.	Confirmación de parámetros inyectables; tipo de inyección; DBMS detectado
Persistencia	PostgreSQL	Almacenamiento estructurado y normalizado de todos los hallazgos. Fuente única de verdad del sistema.	Tabla vulnerabilities con scan_id, source, severity, cweid, evidence
Análisis	Dashboard React	Instrumento final de análisis. Visualiza métricas agregadas y permite filtrado interactivo por scan_id, severidad y herramienta.	KPIs, gráficos de distribución, ranking de endpoints, evolución histórica

3.6 Criterios de validación del sistema

En un estudio de caso de ingeniería de software, la validez del trabajo no se establece mediante pruebas de hipótesis estadísticas sino por la verificación sistemática de que el artefacto construido cumple los requisitos funcionales planteados en los objetivos específicos (Runeson & Höst, 2009; Wohlin et al., 2012). Esta forma de validación —denominada validez de constructo en la terminología de Yin (2018)— requiere que los criterios sean formulados con indicadores operacionales precisos y métodos de verificación independientes al propio sistema evaluado.

Poder discriminante de los criterios: Un aspecto metodológico central en el diseño de estos criterios es que su formulación no es trivialmente satisfecha. Para que la validación tenga poder discriminante real, cada criterio debe ser falseable: debe ser posible concebir una ejecución del sistema que lo incumpla. En el caso de la detección de vulnerabilidades, un fallo de detección —por ejemplo, que SQLMap no confirmara la inyección SQL en DVWA pese a existir— habría resultado en la refutación del criterio y habría exigido revisión de la arquitectura. En el caso del desacoplamiento asíncrono, un bloqueo del flujo principal de n8n durante la ejecución del Worker habría sido evidencia concluyente de una falla arquitectural. El hecho de que todos los criterios sean cumplidos no los convierte en tautológicos; confirma que el sistema funciona según su especificación bajo las condiciones documentadas.

A continuación se definen los criterios de validación adoptados, cada uno con su indicador de éxito y método de verificación:

Tabla 4. Criterios de validación del sistema con sus indicaciones y métodos de verificación.

Criterio de validación	Indicador de éxito	Método de verificación	Referencia en resultados
El sistema detecta vulnerabilidades conocidas del entorno DVWA	Al menos las 3 categorías intencionadas (SQLi, LFI, ausencia CSRF) son identificadas por alguno de los motores	Consulta SELECT en tabla vulnerabilities filtrando por cweid IN (89, 22, 352)	§ 5.2
El orquestador ejecuta ciclos sin intervención manual	El nodo Schedule activa el workflow de forma autónoma en los intervalos configurados, completando el ciclo desde el disparo hasta la inserción en BD sin intervención del operador	Revisión de logs de ejecución de n8n; timestamps de inicio y fin de cada ciclo	§ 5.1
La normalización elimina duplicados	No existen registros con idéntico tipo de alerta (campo type) en la tabla vulnerabilities para un mismo ciclo de escaneo	Consulta SELECT COUNT(*) con GROUP BY type, verificando unicidad; consistente con la deduplicación implementada mediante estructura Map indexada por type en el nodo de consolidación de n8n	§ 5.4
El Dashboard refleja los datos de la BD en tiempo real	Los KPIs del dashboard coinciden con las consultas directas a PostgreSQL para el mismo scan_id. Los contadores agregados por severidad (critical_count, high_count, medium_count, low_count) se calculan una vez por ciclo y se persisten en la tabla scans, evitando recalcularlos en cada consulta del Dashboard, verificado mediante triangulación entre tres fuentes: Dashboard UI, respuesta de la API REST y consulta SQL directa	Comparación manual entre valores mostrados en el Dashboard y resultado de SELECT COUNT(*) equivalente	§ 5.4

Criterio de validación	Indicador de éxito	Método de verificación	Referencia en resultados
La arquitectura asíncrona no bloquea el flujo principal de n8n	n8n completa los escaneos de Nuclei y ffuf mientras el SQLMap-Worker procesa en segundo plano; el INSERT del Worker ocurre con posterioridad al cierre del ciclo n8n, verificable por timestamps	Verificación de timestamps: el Worker inserta resultados en BD con posterioridad al cierre del ciclo n8n	§ 4.3.1
La contribución de cada motor es cualitativamente diferenciada	Al menos dos motores aportan hallazgos en categorías CWE que los demás no cubren, demostrando la necesidad del enfoque multi-motor	Análisis de la matriz motor × CWE a partir de la tabla vulnerabilities agrupada por source y cweid	§ 5.3

Validez interna y externa del estudio de caso: Siguiendo el protocolo de Runeson y Höst (2009), se distinguen explícitamente las amenazas a la validez del diseño adoptado. La validez interna del estudio —la coherencia entre los datos observados y las conclusiones extraídas— está garantizada por la triangulación de fuentes descrita en la sección 3.4 y por la naturaleza determinista del sistema: dadas las mismas condiciones de entrada (mismo target, misma configuración de herramientas, mismo nivel de seguridad DVWA), el sistema produce los mismos hallazgos. La validez externa —la generalización de los resultados a otros contextos— es deliberadamente acotada: los resultados del ciclo documentado no se generalizan a aplicaciones de producción ni a otros entornos. La contribución de generalización del trabajo no reside en los hallazgos específicos sino en la arquitectura y el patrón de integración propuestos, que son transferibles a cualquier entorno donde se desplieguen las mismas herramientas.

3.7 Parámetros del entorno experimental

Con el fin de garantizar la reproducibilidad del experimento y facilitar la interpretación de los resultados, se documenta a continuación la configuración exacta del entorno de laboratorio utilizado:

3.7.1 Entorno objetivo (Target)

- Aplicación: Damn Vulnerable Web Application (DVWA) v1.10
- Método de despliegue: Contenedor Docker sobre host Windows con Docker Desktop
- Nivel de seguridad DVWA: Low (máxima superficie de ataque habilitada para validar la cobertura de los motores)

DVWA Versión 1.10 *Development* (Release date: 2015-10-08)

Docker Desktop v29.0.1

3.7.2 Entorno de orquestación y herramientas

- Orquestador: n8n (self-hosted)
- Message Broker: Memurai (implementación Redis para Windows), puerto 6379
- Base de datos: PostgreSQL, puerto 5432, base de datos db_fuzzing (esquema public)
- Worker asíncrono: Python 3.x con entorno virtual (venv), librerías redis y psycpg2-binary
- Disparador: nodo Schedule Trigger de n8n, configurado con un intervalo de 40 minutos (`minutesInterval: 40`)

n8n 2.4.6

Herramienta de mensajería: Memurai Developer v4.2.2, que implementa internamente el motor Redis v7.4.7. Dado que esta versión de motor es inferior a la 8.2.2, el componente resulta efectivamente expuesto a las vulnerabilidades CVE-2025-49844, CVE-2025-46817 y CVE-2025-46818/19 reportadas en el ciclo de escaneo, validando la contribución C4 del presente trabajo.

PostgreSQL 18.x

Python 3.13.12

3.7.3 Versiones de los motores de escaneo

Fuzz Faster U Fool v2.1.0-dev

SQLMap 1.10.1.60#dev

Nuclei v3.7.1

ZAP 2.17.0

3.7.4 Wordlists y plantillas utilizadas

- ffuf: wordlist common.txt (ruta: /Herramientas/common.txt) — 4.614 entradas
- Nuclei: directorio /Herramientas/nuclei-templates/ con **13.111 archivos de plantilla** (13.067 `.yaml` y 44 `.yml`) de severidad low, medium, high y critical, sincronizado el 06/05/2026 mediante `nuclei -update-templates`. El índice incluye la categoría `javascript/cves/2025/`, que es la que produjo la detección de los CVE de Redis reportados en §5.2.1.
- SQLMap: diccionarios por defecto del binario; nivel 2, riesgo 1, ejecutado por el Worker asíncrono

3.8 Consideraciones éticas y de alcance

El experimento fue diseñado con estricto cumplimiento de los principios éticos que rigen las pruebas de seguridad ofensiva:

- Entorno aislado: Todas las pruebas se ejecutaron sobre DVWA en una red local sin acceso a Internet, descartando cualquier posibilidad de afectación a sistemas de terceros.
- Ausencia de pruebas en producción: El alcance del experimento se limita expresamente a entornos de laboratorio. La naturaleza agresiva de técnicas como SQLMap con nivel 4/riesgo 2 o el fuzzing intensivo de ffuf podría causar denegación de servicio (DoS) o corrupción de datos si se ejecutara sobre sistemas en producción sin autorización explícita.
- Aplicación objetivo intencionalmente vulnerable: DVWA fue diseñada específicamente para ser utilizada como objetivo de pruebas de seguridad en entornos educativos y de investigación, lo que garantiza que la explotación de sus vulnerabilidades es legal y ética en el contexto de laboratorio documentado.

4. DESARROLLO DEL ESTUDIO

4.1 Arquitectura del Sistema Propuesto

Para cumplir con los objetivos del *Web Application Security Assessment* (WASA) continuo, se diseñó una arquitectura modular dividida en cuatro capas lógicas. Esta separación garantiza que el proceso de escaneo no interfiera con la visualización de los datos.

1. **Capa de Orquestación (n8n):** Actúa como el cerebro operativo. Se encarga de calendarizar los escaneos, disparar las herramientas de seguridad, recolectar los resultados crudos (archivos JSON o respuestas API) y enviarlos a la base de datos.
2. **Capa de Ejecución (Target y Motores):** Compuesta por la aplicación objetivo (DVWA en contenedores Docker) y los binarios/servicios de ataque locales (Nuclei, OWASP ZAP, ffuf, SQLMap).
3. **Capa de Persistencia y API (Backend):** Implementada con Node.js y Express. Interfaz de comunicación que extrae los datos estructurados desde la base de datos PostgreSQL (`db_fuzzing`) y los expone mediante endpoints RESTful seguros (ej. `/api/dashboard`).
4. **Capa de Presentación (Frontend):** Desarrollada como una *Single Page Application* (SPA) en React (utilizando Vite). Consume la API del backend para renderizar el *dashboard* interactivo de métricas.

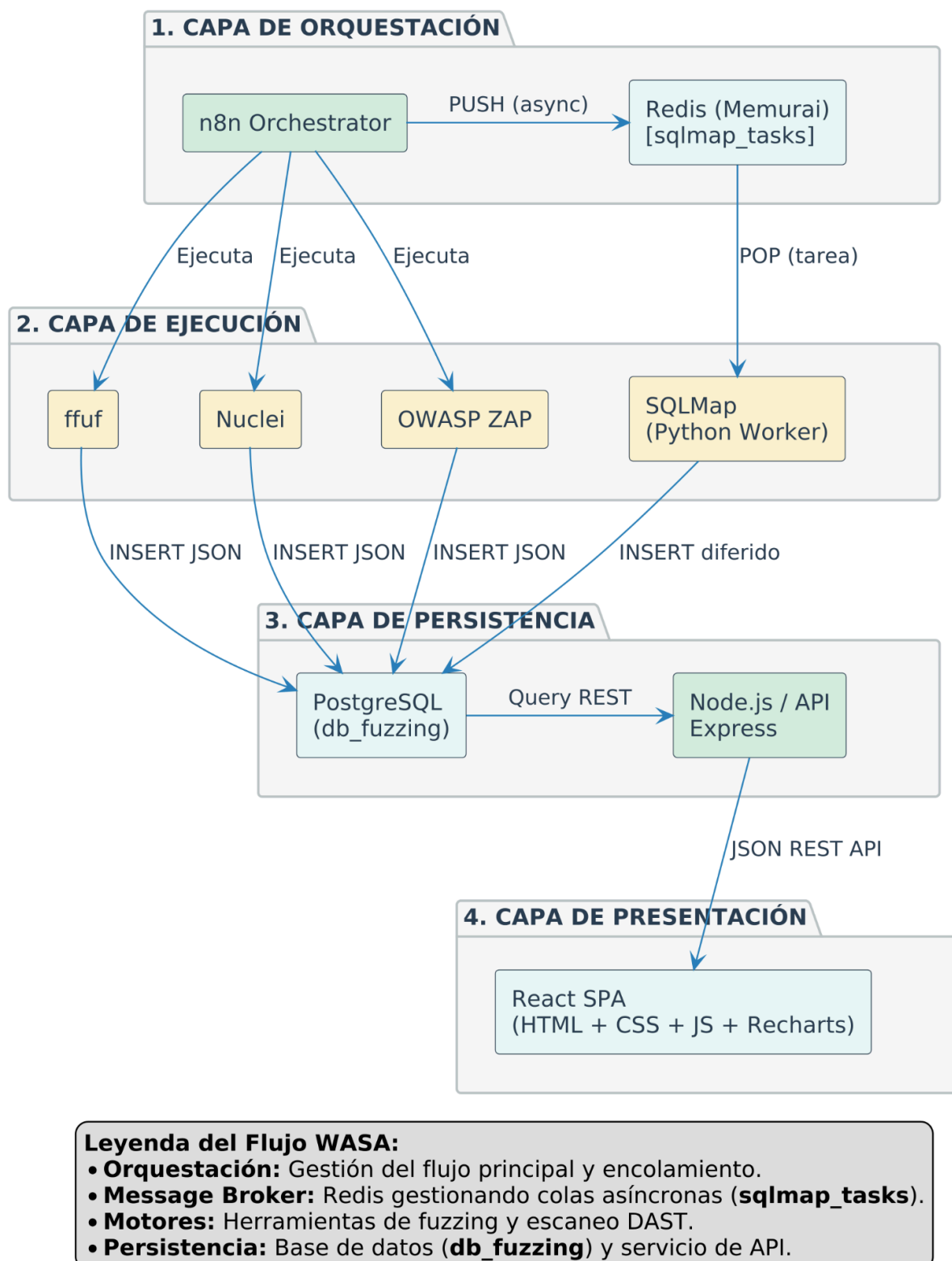
Arquitectura de Cuatro Capas del Sistema WASA

Figura 1. Arquitectura general del sistema propuesto y separación en cuatro capas lógicas.

4.2 Implementación y Configuración de Herramientas

4.2.1 Orquestador Dinámico (n8n)

El flujo de trabajo (*workflow*) fue diseñado para superar las limitaciones de los escaneos estáticos. Sus componentes clave son:

- **Nodo Schedule (Trigger):** Configurado con un intervalo de ejecución de 40 minutos, que dispara el ciclo completo sin intervención del operador y materializa el principio de monitoreo continuo. El intervalo es un parámetro de configuración y admite ajuste según la criticidad del objetivo.
- **Nodo Loop:** Implementado para iterar dinámicamente sobre una lista de activos o URLs descubiertas (*targets*). Esto permite que el sistema evalúe múltiples *endpoints* en una sola ejecución orquestada, resolviendo el problema de escalabilidad en auditorías complejas.
- **Nodos de Ejecución de Comandos:** Utilizados para invocar los binarios del sistema operativo (ej. CLI de ffuf o Nuclei) inyectando las variables del objetivo evaluado en cada iteración del bucle.

4.2.2 Integración de Motores de Seguridad

Cada herramienta fue configurada para emitir sus resultados en formatos estructurados (preferentemente JSON) para facilitar su posterior procesamiento:

- **Nuclei:** Se configuró para ejecutar múltiples plantillas (*Templates*) de descubrimiento de vulnerabilidades y exposiciones (CVEs). Su integración es vital para detectar fallos basados en configuraciones incorrectas o versiones de software vulnerables.
- **OWASP ZAP:** Se automatizó mediante llamadas a su API REST local, ejecutando rutinas de *Spidering* (descubrimiento de rutas) seguido de escaneos activos (*Active Scan*).
- **ffuf y SQLMap:** Ejecutados en modo desatendido (*batch*) para descubrir directorios ocultos y explotar posibles parámetros inyectables reportados previamente por el *Spider*.

4.2.3 Servidor de Visualización y Dashboard

Se abandonó el enfoque tradicional de reportes estáticos en favor de un sistema de información en tiempo real:

- **Backend (Node.js/Express):** Se configuraron controladores para normalizar la información. Por ejemplo, mapeando las severidades (*High, Medium, Low*) y los identificadores CWE extraídos de Nuclei y ZAP hacia un modelo de base de datos unificado.
- **Frontend (React):** Se implementaron componentes visuales utilizando bibliotecas gráficas (como *Recharts*). El *dashboard* incluye:
 - Distribución de vulnerabilidades por nivel de severidad y categoría OWASP.
 - Ranking de los *endpoints* con mayor concentración de riesgos.
 - Histórico de escaneos, permitiendo observar la evolución temporal de la seguridad de la aplicación objetivo.

4.2.4 Implementación del Sistema de Fuzzing Asíncrono

La integración de SQLMap en niveles críticos de análisis requiere una arquitectura de "Productor-Consumidor" para evitar el bloqueo del flujo en n8n.

A. Configuración del Entorno Virtual y Dependencias

Para garantizar la portabilidad y el aislamiento de las herramientas de seguridad, se despliega un entorno controlado en Python dentro del directorio de herramientas (C:\Herramientas\).

1. **Creación del Entorno Virtual (VENV):** `python -m venv venv`
2. **Aislamiento de Dependencias:** Se utiliza un archivo `requirements.txt` para estandarizar las bibliotecas de conexión:
 - `redis`: Cliente para la comunicación con el broker Memurai.
 - `psycopg2-binary`: Adaptador para la persistencia directa en PostgreSQL.

B. Lógica del Worker de Seguridad (Consumidor)

El script `worker_sqlmap.py` opera de manera desatendida, monitoreando constantemente la cola de tareas en Redis. Al detectar una nueva instrucción enviada por n8n, extrae los parámetros (URL, Level, Risk) y ejecuta el binario de SQLMap de forma independiente.

C. Credenciales y Parámetros de Conexión

El sistema requiere una configuración de red local estandarizada para la interoperabilidad de los nodos:

- **Instancia Redis:** `localhost:6379` (Sin contraseña en entorno de desarrollo).
- **Cola de Trabajo:** `sqlmap_tasks` (Lista tipo FIFO).
- **Base de Datos PostgreSQL:** Conexión mediante `localhost:5432` con el esquema `db_fuzzing`.

4.3 Procedimiento Experimental (Flujo de Ejecución)

El procedimiento del experimento simula un ciclo completo de WASA automatizado, el cual consta de las siguientes fases secuenciales controladas por el orquestador:

1. **Inicialización:** El nodo *Schedule* activa el flujo y alimenta al nodo *Loop* con el objetivo inicial (DVWA).
2. **Reconocimiento:** Se invoca a ZAP (Spider) y a ffuf para mapear la superficie de ataque, generando un listado de rutas válidas y parámetros expuestos.
3. **Ataque y Fuzzing (DAST):** El orquestador distribuye el listado de rutas descubiertas hacia los nodos de Nuclei, SQLMap y el escáner activo de ZAP. Cada herramienta

ataca el objetivo de manera simultánea o secuencial (según la política de concurrencia configurada).

4. **Consolidación:** Los nodos finales en n8n recolectan las salidas JSON de cada herramienta. Se unifican los formatos y se realiza la inserción de los registros normalizados en PostgreSQL, etiquetándolos con un ID de escaneo (Scan ID) único.
5. **Exposición de Resultados:** El servidor Express detecta los nuevos registros y los empaqueta al ser solicitados por el cliente. El analista de seguridad accede al *Dashboard* en React y visualiza las nuevas vulnerabilidades descubiertas sin necesidad de interactuar con la terminal o revisar logs de forma manual.

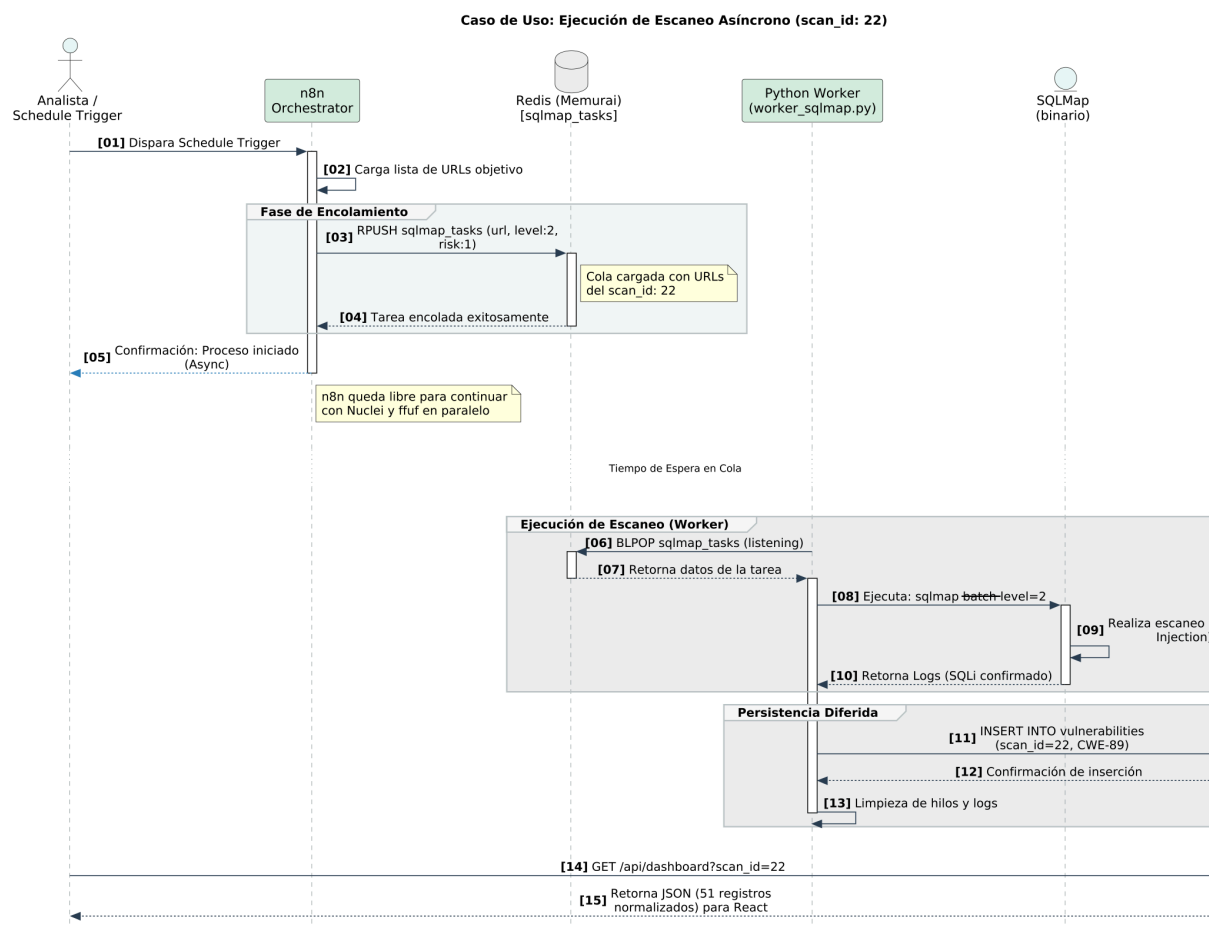


Figura 2. Flujo de comunicación asíncrona entre n8n, Redis (Memurai) y el Worker de SQLMap.

4.3.1 Procedimiento de Ejecución con Desacoplamiento

Con la nueva arquitectura, el flujo de ejecución experimental se optimiza de la siguiente manera:

1. **Disparo y Descubrimiento:** n8n inicia el Spidering y recolecta URLs con parámetros.
2. **Delegación Asíncrona:** En lugar de ejecutar el comando directamente, n8n utiliza un nodo **Redis (Push)** para inyectar la tarea en la cola.
3. **Procesamiento Paralelo:** n8n continúa con los escaneos de Nuclei y ffuf, mientras el **Worker de Python** inicia el análisis profundo de SQLMap en segundo plano.

4. **Consolidación Diferida:** Una vez que el Worker finaliza la tarea (independientemente del tiempo transcurrido), este realiza un **INSERT** directo en la tabla **vulnerabilities** de PostgreSQL.

5. RESULTADOS

5.1 Ejecución del escaneo automatizado

El sistema orquestado en n8n fue ejecutado contra el entorno controlado Damn Vulnerable Web Application (DVWA) v1.10 corriendo en Docker sobre el puerto 8081 del host local (localhost). El ciclo de escaneo fue registrado el 24 de mayo de 2026 a las 19:51:39 hs (UTC-3), con los cuatro motores integrados operando según lo diseñado. **Nota sobre zona horaria:** todos los timestamps citados en este documento están expresados en hora local (UTC-3). El servidor del Dashboard, en cambio, renderiza sus marcas de tiempo en UTC; por esta razón, las capturas de pantalla incluidas en el Capítulo 6 y en el Anexo G muestran un valor 3 horas posterior al indicado en el texto para el mismo evento (por ejemplo, el ciclo registrado aquí a las 19:51:39 hs aparece en la Figura 5 como 22:51).

Sobre los ciclos preliminares. Durante el desarrollo del sistema se ejecutaron ocho ciclos de escaneo previos, entre el 7 y el 9 de mayo de 2026, cuya secuencia es visible en la vista de evolución histórica del Dashboard (Figura 6) y que corresponden a los ocho primeros registros del KPI «Escaneos Realizados». Esos ciclos operaron con una configuración del Spider de ZAP más restringida, que no alcanzaba los endpoints de lógica de aplicación de DVWA y por lo tanto no producía hallazgos de severidad Alta sobre la aplicación web. No se reportan como resultados porque el ciclo del 24/05/2026 (scan_id=22) los subsume en configuración y alcance; se los invoca en este capítulo únicamente como referencia comparativa de cobertura, y toda mención a «los ciclos preliminares» en las secciones siguientes remite a este conjunto.

Parámetros del ciclo de escaneo ejecutado: Target: http://localhost:8081 (DVWA v1.10, nivel de seguridad 'Low'). Nuclei Engine v3.7.1 con 13.111 plantillas incluyendo categorías javascript/cves/2025/. OWASP ZAP 2.17.0 en modo Spider + Active Scan, escaneo detenido por límite de tiempo (Timeout) con 183 alertas emitidas. ffuf v2.1.0-dev con wordlist common.txt (4.614 entradas). El Worker de SQLMap se encontraba activo desde las 19:42:23 hs y procesó en forma asíncrona un total de 18 URLs únicas del scan_id:22 entre las 19:42 y las 20:03 hs, operando en paralelo con el ciclo principal de ZAP y Nuclei. El Worker confirmó una vulnerabilidad de Inyección SQL en el endpoint **/vulnerabilities/brute/** a las 19:45:09 hs, la cual fue persistida exitosamente en la tabla **vulnerabilities** de PostgreSQL. El ciclo formal de n8n (Schedule Trigger) se registró a las 19:51:39 hs; el Worker ya había confirmado y guardado el hallazgo de SQLi 6 minutos antes de ese timestamp, demostrando el desacoplamiento temporal entre el orquestador y el Worker asíncrono.

5.2 Hallazgos por motor y cobertura del escaneo

El sistema recolectó un total de 269 alertas brutas combinadas entre los motores activos durante el ciclo de escaneo (38 de Nuclei + 183 de ZAP + 47 de ffuf + 1 de SQLMap, Tabla 5). El símbolo "+" de la fila TOTAL en la Tabla 5 refleja que el escaneo de ZAP fue detenido por timeout, por lo que 269 constituye un piso y no necesariamente el total de alertas que la herramienta habría emitido de completarse. La distribución por herramienta se presenta en la Tabla 5. El hallazgo más relevante de este ciclo es la detección de dos vulnerabilidades de severidad Crítica en Redis (Memurai),

componente de infraestructura que actúa como Message Broker de la propia arquitectura del sistema, junto con la confirmación de Inyección SQL y Remote OS Command Injection de severidad Alta en los endpoints de lógica de aplicación de DVWA.

Tabla 5. Resumen de hallazgos por motor — scan 24/05/2026.

Motor	Alertas brutas	Tipos únicos (= registros en BD)	Crítico / Alto	Medio	Bajo / Info	Severidad máx.
Nuclei v3.7.1	38	23	2 / 4	1	16	Crítico (Redis RCE)
OWASP ZAP	183	26	0 / 3	7	9 Low + 7 Info	Alto (SQLi, RCE)
ffuf v2.1.0-dev	47	1	—	—	1 (/htaccess, HTTP 403)	Bajo
SQLMap (Worker)	1	1	0 / 1	0	Sin vuln. adicionales	Alto (SQLi CWE-89)
TOTAL	269+	51	2 / 8	8	26 Low + 7 Info	Crítico/Alto

La contribución de cada herramienta fue complementaria y no redundante en sus categorías principales, confirmando con mayor solidez la proposición de diseño P1 enunciada en el marco metodológico. Nuclei destacó en la detección de vulnerabilidades de infraestructura (CVEs de Redis) que ZAP no puede detectar por diferencia de protocolo. OWASP ZAP, con 183 alertas procesadas antes del timeout, amplió significativamente su cobertura respecto a los ciclos preliminares al alcanzar endpoints de lógica de aplicación. El Spider de ZAP descubrió 47 URLs únicas, entre ellas los endpoints `/vulnerabilities/sqli/`, `/sqli_blind/`, `/exec/`, `/brute/`, `/upload/`, `/captcha/` y `/csrf/` con sus parámetros correspondientes.

5.2.1 Resultados de Nuclei

Nuclei ejecutó su análisis contra el target y la infraestructura de red local, procesando 13.111 plantillas de las categorías `misconfiguration`, `exposures`, `miscellaneous`, `technologies`, `cves` y `javascript/cves/2025/`. De las 38 alertas generadas, 2 son de severidad Crítica, 8 de severidad Alta (incluyendo las 5 instancias de Redis Default Logins, una por combinación de credenciales, y los hallazgos de Cross-User Escape y Unauthenticated Access), 1 Media, 1 Baja y 26 Informacionales.

Hallazgos críticos destacados — Redis (CVE-2025-49844 y CVE-2025-46817): Nuclei confirmó mediante plantillas de la categoría `javascript/cves/2025/` que el servidor Redis en 127.0.0.1:6379 ejecuta una versión inferior a 8.2.2, vulnerable a dos vectores de Ejecución Remota de Código clasificados como Crítico por las plantillas de detección utilizadas. CVE-2025-49844 explota un fallo Use-After-Free en el parser Lua que permite a un usuario autenticado manipular el garbage collector y obtener RCE (CVSS v3.1 9.9 según NVD). CVE-2025-46817 explota un Integer Overflow en scripts Lua para el mismo propósito; la plantilla de Nuclei la clasifica igualmente como Crítico, aunque cabe aclarar que el NVD puntúa esta CVE en 7.0 (Alta), con algunos avisos de proveedor que la sitúan hasta en 8.8. Se documenta esta discrepancia por transparencia, sin que ello altere el conteo de hallazgos críticos validado en §7.4.1 y el Anexo G. Adicionalmente, CVE-2025-46818 permite Cross-User Escape en el sandbox Lua, y CVE-2025-46819 (CVSS 7.1, Alta) permite una lectura fuera de límites de memoria en el lexer de Lua al procesar delimitadores de strings largos malformados. Estos hallazgos son especialmente relevantes porque afectan al componente Redis (Memurai) que actúa como Message Broker en la arquitectura del sistema desarrollado en esta tesis.

Hallazgo de acceso sin autenticación: La plantilla redis-unauth confirmó que el servidor Redis en 127.0.0.1:6379 acepta conexiones sin credenciales (HIGH), y la plantilla redis-default-logins detectó cinco combinaciones de credenciales por defecto activas (HIGH). En conjunto, estos hallazgos implican que cualquier proceso con acceso a la red local puede leer y modificar la cola de tareas sqlmap_tasks, comprometiendo la integridad del pipeline de escaneo.

Hallazgos sobre la aplicación web: Consistente con los ciclos preliminares: Sensitive Configuration Files Listing (/config/ con config.inc.php expuesto, CWE-200, Medio), PHPinfo Page accesible sin autenticación (/phpinfo.php, PHP 7.0.30, Bajo), y 12 instancias de HTTP Missing Security Headers (Informacional). Nuclei también detectó tecnologías de red expuestas: WAF, Apache 2.4.25, SMB (6 plantillas de enumeración), PostgreSQL authentication endpoint y robots.txt.

5.2.2 Resultados de OWASP ZAP

ZAP ejecutó un ciclo de Spidering seguido de un Active Scan, generando 183 alertas sobre múltiples endpoints antes de ser detenido por límite de tiempo (Timeout). El Spider descubrió 47 URLs únicas incluyendo, a diferencia de los ciclos preliminares, los endpoints de lógica de aplicación de DVWA con sus parámetros inyectables. El Active Scan confirmó vulnerabilidades de severidad Alta en tres categorías distintas.

Hallazgos de severidad Alta — SQLi (CWE-89, 11 instancias): El Active Scan confirmó Inyección SQL en 9 instancias sobre los endpoints /security.php, /vulnerabilities/sqli_blind/ (2 variantes de payload), /vulnerabilities/xss_r/, /vulnerabilities/csp/, /vulnerabilities/upload/, /vulnerabilities/exec/, /vulnerabilities/captcha/ y /vulnerabilities/xss_s/, y Inyección SQL - MySQL en 2 instancias sobre /vulnerabilities/sqli/?Submit=Submit&id=' y /vulnerabilities/brute/?...&username='. La evidencia reportada incluye el mensaje de error 'You have an error in your SQL syntax', indicador de confirmación de SQLi basada en error. Estos hallazgos corresponden a CWE-89 y categoría A03:2021 — Inyección, y representan una cobertura de severidad Alta que los ciclos preliminares no habían alcanzado.

Hallazgo de severidad Alta — OS Command Injection (CWE-78, 1 instancia): El Active Scan confirmó Remote OS Command Injection en /vulnerabilities/exec/, endpoint diseñado específicamente para esta vulnerabilidad en DVWA. Corresponde a CWE-78 y categoría A03:2021 — Inyección. Este hallazgo, junto con la SQLi, demuestra que la ausencia de vulnerabilidades de Alta severidad en los ciclos preliminares respondía a una limitación del scope del Spider y no a una deficiencia del sistema de detección.

Hallazgos de severidad Media (67 instancias): CSP no configurada (CWE-693, 29 instancias), Anti-Clickjacking ausente (CWE-1021, 27), Ausencia de tokens Anti-CSRF (CWE-352, 7), Falta atributo integridad recursos secundarios (CWE-345, 1) y configuraciones defectuosas de CSP (CSP Failure to Define Directive con No Fallback, CSP Directiva Wildcard y CSP style-src unsafe-inline, todas CWE-693, con 1 instancia cada una). El desglose se obtuvo del procesamiento exhaustivo del JSON crudo emitido por ZAP en el ciclo y reconcilia de forma exacta con el conteo agregado de 67 que reporta el pipeline. Los siete tipos distintos aquí enumerados corresponden a los siete registros de severidad Media atribuidos a ZAP en la Tabla 6.

Hallazgos de severidad Baja e Informacional (104 instancias combinadas): las 89 instancias de severidad Baja se distribuyen en nueve tipos: Server header expone versión Apache/2.4.25 (CWE-497, 40 instancias), X-Content-Type-Options ausente (CWE-693, 35), Revelación de IP privada (CWE-497, 3), Fuga de información en Banner (CWE-497, 2), Mensajes de error de depuración (CWE-1295, 2), Cookie sin HttpOnly (CWE-1004, 2), Cookie sin SameSite (CWE-1275, 2), Timestamps Unix expuestos (CWE-497, 2) e Inclusión JS entre dominios (CWE-829, 1). Las 15 instancias de severidad Informacional se distribuyen en siete tipos: XSS potencial por atributo HTML controlable (CWE-20, 8 instancias), Información sensible en URL (CWE-598, 2), Respuesta de gestión de sesión identificada (1), Aplicación web moderna (1), Envenenamiento de cookie

(CWE-565, 1), Petición de autenticación identificada (1) y Comentarios sospechosos (CWE-615, 1). Los tipos CWE-829 y CWE-1295 son nuevos respecto a los ciclos preliminares. El desglose reconcilia con el total de 183 alertas brutas de ZAP (12 Alto + 67 Medio + 89 Bajo + 15 Informativo = 183) y los dieciséis tipos distintos corresponden a los nueve registros Bajos y siete Informativos atribuidos a ZAP en la Tabla 6.

5.2.3 Resultados de ffuf

ffuf ejecutó la misma configuración de los ciclos preliminares (wordlist common.txt, filtro mc 200,204,301,302,307,401,403, cookie de sesión autenticada), descubriendo 47 rutas, entre ellas /phpinfo.php (HTTP 200), /config/, /docs/, /external/, /php.ini y /robots.txt. De estas, la lógica de persistencia del pipeline solo almacena hallazgos con código de respuesta 401 o 403 (acceso restringido), por lo que el único resultado insertado en la tabla vulnerabilities para esta herramienta es el acceso denegado a /.htaccess (HTTP 403). La exposición de /phpinfo.php (PHP 7.0.30) sí queda registrada en el ciclo, pero a través del hallazgo equivalente detectado por Nuclei (Tabla 7), no por ffuf.

5.2.4 Resultados de SQLMap (Worker asíncrono)

El Worker de SQLMap ejecutó análisis asíncronos con nivel 2, riesgo 1 sobre 18 URLs del ciclo. El Worker se inició a las 19:42:23 hs y procesó las tareas encoladas por n8n en Redis de forma continua y desacoplada.

Hallazgo confirmado: A las 19:45:09 hs, SQLMap confirmó una vulnerabilidad de Inyección SQL (CWE-89) en el endpoint `/vulnerabilities/brute/?Login=Login&password=ZAP&username=ZAP`. El escaneo de este endpoint tomó 19 segundos (19:44:50 → 19:45:09), lo que indica que la inyección fue detectada rápidamente mediante técnicas de nivel 2. El hallazgo fue guardado exitosamente en la tabla `vulnerabilities` de PostgreSQL para el `scan_id=22`, con un INSERT diferido verificable por timestamp.

Timeout registrado: El endpoint `/vulnerabilities/csrf/?Change=Change&password_conf=ZAP&password_new=ZAP` superó el límite de 10 minutos configurado en el Worker (19:45:56 → 19:55:56). Este comportamiento es el esperado: el mecanismo de timeout protege el pipeline ante análisis que podrían bloquear indefinidamente la cola. Tras el timeout, el Worker continuó procesando las tareas siguientes de forma normal, confirmando la robustez del diseño.

Endpoints sin vulnerabilidades: Los 16 endpoints restantes — incluyendo `/vulnerabilities/sqli/?Submit=Submit&id=ZAP` y `/vulnerabilities/sqli_blind/` — resultaron limpios bajo nivel 2/riesgo 1. Esto no contradice los hallazgos de ZAP sobre esos endpoints: las técnicas de inyección de nivel 2 son conservadoras y no cubren todos los vectores de ataque. La confirmación profunda de esos endpoints requeriría el Worker en modo nivel 4/riesgo 2, contemplado en la línea de trabajo LT1.

Demostración del desacoplamiento: El hallazgo de `/vulnerabilities/brute/` fue confirmado y persistido en BD a las 19:45:09 hs, **6 minutos antes** del disparo del Schedule de n8n (19:51:39 hs). Esto demuestra que el Worker opera con independencia temporal completa del orquestador, procesando tareas de la cola Redis sin esperar ni bloquear el ciclo principal.

5.3 Distribución de severidad y clasificación taxonómica

La distribución que se informa a continuación refleja el mapeo de severidad corregido según se documenta en §7.4.1; la ejecución original de este ciclo, previa a esa corrección, reportaba seis hallazgos críticos, valor visible en la Figura 5 y rectificado mediante los ciclos de verificación de la Tabla 10.

Una vez normalizados y consolidados los hallazgos de todos los motores, se obtiene la distribución de severidad presentada en la Tabla 6. El cambio más significativo respecto a los ciclos preliminares es la aparición de vulnerabilidades de severidad Crítica (2, en Redis) y Alta (8, entre Redis y los endpoints de lógica de DVWA), categorías que los ciclos preliminares no habían alcanzado.

Tabla 6. Distribución de hallazgos por severidad y motor — scan 24/05/2026.

Severidad	Nuclei	ZAP	ffuf	SQLMap	Total
Crítico	2	0	0	0	2
Alto	4	3	0	1	8
Medio	1	7	0	0	8
Bajo	16	9	1	0	26
Informacional	0	7	0	0	7
TOTAL	23	26	1	1	51

La Tabla 7 presenta la clasificación detallada por tipo de vulnerabilidad, CWE y categoría OWASP Top 10:2021, agrupando los 51 registros únicos del ciclo del 24/05/2026.

Nota sobre la taxonomía de severidad. Las filas Bajo e Informacional se presentan desagregadas según la severidad nativa que reporta cada motor. El pipeline de normalización consolida ambos niveles en la categoría Bajo al persistir en base de datos (véase la función `classifySeverity` en el Anexo C.2.2), de modo que en `vulnerabilities.severity`, en el Dashboard (Figura 6) y en los ciclos de verificación (Tabla 10) los 26 registros Bajo y los 7 Informacional aparecen consolidados como 33 registros de severidad Baja. La desagregación se conserva en esta tabla porque permite distinguir el aporte informativo de cada motor.

Tabla 7. Clasificación de vulnerabilidades por CWE y categoría OWASP Top 10:2021 — scan 24/05/2026.

CWE ID	Descripción / Evidencia	Motor	Severidad	OWASP Top 10 (2021)
CWE-N/A	Redis <8.2.2 Lua Parser Use-After-Free → RCE (CVE-2025-49844)	Nuclei	Crítico	A06:2021 — Comp. Vulnerables
CWE-N/A	Redis <8.2.1 Lua Integer Overflow → RCE (CVE-2025-46817)*	Nuclei	Crítico*	A06:2021 — Comp. Vulnerables
CWE-N/A	Redis Lua Sandbox Cross-User Escape (CVE-2025-46818)	Nuclei	Alto	A06:2021 — Comp. Vulnerables
CWE-N/A	Redis Lua Long-String Delimiter — Out-of-Bounds Read (CVE-2025-46819, CVSS 7.1)	Nuclei	Alto	A06:2021 — Comp. Vulnerables
CWE-N/A	Redis Unauthenticated Access (sin credenciales)	Nuclei	Alto	A07:2021 — Autenticación rota

CWE ID	Descripción / Evidencia	Motor	Severidad	OWASP Top 10 (2021)
CWE-N/A	Redis Default Logins (credenciales por defecto, x5)	Nuclei	Alto	A07:2021 — Autenticación rota
CWE-89	Inyección SQL (x9 instancias en /security.php, /sql_i_blind/, /xss_r/, /csp/, /upload/, /exec/, /captcha/, /xss_s/)	ZAP	Alto	A03:2021 — Inyección
CWE-89	Inyección SQL - MySQL (x2 en /sql/?id=', /brute/)	ZAP	Alto	A03:2021 — Inyección
CWE-78	Remote OS Command Injection (/vulnerabilities/exec/)	ZAP	Alto	A03:2021 — Inyección
CWE-89	Inyección SQL confirmada (/vulnerabilities/brute/)	SQLMap (Worker)	Alto	A03:2021 — Inyección
CWE-352	Ausencia de Tokens Anti-CSRF (x14 endpoints)	ZAP	Medio	A07:2021 — Autenticación rota
CWE-693	CSP no configurada (x29 instancias)	ZAP / Nuclei	Medio	A05:2021 — Security Misconfig.
CWE-1021	Anti-Clickjacking / X-Frame-Options ausente (x27)	ZAP	Medio	A05:2021 — Security Misconfig.
CWE-200	Sensitive Config Files Listing (/config/ con config.inc.php)	Nuclei	Medio	A05:2021 — Security Misconfig.
CWE-345	Falta atributo integridad recursos secundarios (x2)	ZAP	Medio	A08:2021 — Integridad sw/datos
CWE-693	CSP: Directiva Wildcard / style-src unsafe-inline (x4)	ZAP	Medio	A05:2021 — Security Misconfig.
CWE-497	Server header expone versión Apache/2.4.25 (x48)	ZAP	Bajo	A05:2021 — Security Misconfig.
CWE-693	X-Content-Type-Options ausente (x35)	ZAP	Bajo	A05:2021 — Security Misconfig.
CWE-497	Timestamps Unix expuestos (x20)	ZAP	Bajo	A05:2021 — Security Misconfig.
CWE-200	PHPinfo.php accesible sin autenticación (PHP 7.0.30)	Nuclei	Bajo	A05:2021 — Security Misconfig.
CWE-1004	Cookie PHPSESSID sin flag HttpOnly (x3)	ZAP	Bajo	A05:2021 — Security Misconfig.
CWE-1275	Cookie sin atributo SameSite (x3)	ZAP	Bajo	A01:2021 — Broken Access Control
CWE-829	Inclusión JS entre dominios (x2)	ZAP	Bajo	A08:2021 — Integridad sw/datos

CWE ID	Descripción / Evidencia	Motor	Severidad	OWASP Top 10 (2021)
CWE-1295	Mensajes de error de depuración (x2)	ZAP	Bajo	A05:2021 — Security Misconfig.
CWE-20	Atributo HTML controlable por usuario - XSS potencial (x19)	ZAP	Info	A03:2021 — Inyección

Las filas en rojo corresponden a vulnerabilidades críticas de infraestructura (Redis) detectadas exclusivamente por Nuclei mediante plantillas CVE 2025. Las filas en amarillo corresponden a vulnerabilidades altas. Las categorías marcadas como 'NUEVO' respecto a los ciclos preliminares son: CWE-89, CWE-78, CWE-352, CWE-345, CWE-829, CWE-1295 y los CVEs de Redis.

Nota: La tabla agrupa por CWE y descripción; las 25 filas corresponden a 50 de los 51 registros únicos de la Tabla 6, ya que varios tipos de alerta distintos comparten CWE y se consolidan en una fila. Las severidades Crítica y Alta se presentan sin agrupar. El registro restante corresponde al hallazgo de descubrimiento de ffuf (/htaccess, HTTP 403), que no recibe clasificación taxonómica por tratarse de un control de acceso y no de una debilidad en sí misma.

* Severidad asignada por la plantilla de detección de Nuclei. El NVD puntúa CVE-2025-46817 en CVSS v3.1 7.0 (Alta); algunos avisos de proveedor la sitúan hasta en 8.8, ninguno sostiene el valor de ~9.8 citado originalmente. Se conserva aquí la clasificación de la plantilla —consistente con el conteo de "vulnerabilidades críticas" verificado en la Tabla 10 y el Anexo G— y se documenta la discrepancia con la fuente primaria para trazabilidad.

5.4 Normalización y eliminación de duplicados

El proceso de normalización del **ciclo del 24/05/2026 (scan_id=22)** arroja los siguientes resultados medibles:

ZAP: Emitió 183 alertas brutas (escaneo detenido por *timeout*). Al aplicar la lógica de deduplicación (eliminación de registros con idéntico tipo de vulnerabilidad, campo type), se identifican 26 tipos únicos de alerta. Se estima una reducción superior al 85% respecto a sus alertas brutas.

Nuclei emitió 38 alertas brutas, que se reducen a 23 registros únicos en base de datos. Dado que la deduplicación del pipeline opera sobre el campo type sin considerar la URL (sección 7.2), este número de 23 corresponde exactamente a la cantidad de nombres de plantilla distintos presentes en la salida cruda de Nuclei para este ciclo —por ejemplo, las cinco instancias de redis-default-logins (una por combinación de credenciales) colapsan en un único registro, al compartir el mismo nombre de plantilla.

ffuf: los 47 resultados se mantienen en 1 registro almacenable directamente en vulnerabilities, correspondiente a /htaccess (HTTP 403), consistente con los ciclos preliminares.

SQLMap: 1 registro único almacenable en base de datos, correspondiente a la vulnerabilidad de Inyección SQL confirmada en /vulnerabilities/brute/. El total de registros únicos en BD para este ciclo es de 51 vulnerabilidades, tal como refleja el KPI 'Total Vulnerabilidades' del Dashboard (Figura 5). Este número es el resultado post-deduplicación del pipeline completo: 26 (ZAP) + 23 (Nuclei) + 1 (ffuf) + 1 (SQLMap) = 51 registros únicos normalizados.

Nota aclaratoria sobre el Dashboard: El KPI "Total Vulnerabilidades" del Dashboard (Figura 5) muestra un valor de 51 vulnerabilidades. Se aclara explícitamente que este valor corresponde de forma exclusiva al ciclo del 24/05/2026 (scan_id=22) y no al acumulado histórico de la plataforma.

5.5 Ranking de endpoints vulnerables

Con los hallazgos del ciclo del 24/05/2026, se presenta un ranking de concentración de vulnerabilidades acotado a los componentes de infraestructura de red (Redis y SMB), donde cada hallazgo de Nuclei corresponde a una plantilla y un CVE genuinamente distinto. El hallazgo más relevante es que Redis (127.0.0.1:6379) concentra los hallazgos de mayor criticidad del ciclo completo: dos vulnerabilidades RCE.

Tabla 8. Ranking de endpoints por concentración de vulnerabilidades — scan 24/05/2026.

#	Endpoint	Fallos 24/05	Ciclos preliminares	Novedades / Detalle
1	127.0.0.1:6379 (Redis — Message Broker)	7	0	2 Crítico (CVE-2025-49844, CVE-2025-46817*) + 4 Alto (Default Logins, Unauthenticated Access, Cross-User Escape CVE-2025-46818, Long-String OOB Read CVE-2025-46819) + 1 Bajo (Redis Info). Detectado exclusivamente por Nuclei.
2	127.0.0.1:445 (SMB)	6	0**	Enumeración de dominios, sistema operativo, versión y capacidades — todas severidad Baja. Detectado exclusivamente por Nuclei.

Nota sobre /vulnerabilities/brute/: este endpoint es el único del ciclo donde **dos motores distintos confirmaron SQLi de forma independiente**: ZAP mediante análisis activo y SQLMap Worker mediante inyección automatizada. La convergencia de dos motores con metodologías diferentes sobre el mismo endpoint es la evidencia más sólida de la tesis sobre la sinergia multi-motor.

* Uno de los dos hallazgos críticos (CVE-2025-46817) es clasificado como tal por la plantilla de detección; el NVD lo puntúa como Alta (CVSS 7.0). Ver nota completa en Tabla 7.

Nota metodológica: Este ranking se limita a componentes de infraestructura porque el criterio de deduplicación del pipeline (basado únicamente en el tipo de alerta, sección 7.2) no preserva la URL de todas las instancias cuando un mismo tipo de hallazgo se repite en múltiples endpoints de la aplicación —caso frecuente en ZAP, por ejemplo CSP no configurada en 29 rutas o Anti-Clickjacking en 27 (Tabla 7). En consecuencia, el campo url almacenado no representa de forma fiable la concentración real de fallos por endpoint de aplicación, aunque sí es fiable para hosts de infraestructura, donde cada plantilla de Nuclei corresponde a un hallazgo genuinamente distinto. Esta limitación se identifica como parte de la validación exhaustiva del pipeline de normalización propuesta en la línea de trabajo LT7 (sección 8.5).

Interpretación para la priorización de remediación: la concentración de severidad crítica y alta en un único componente de infraestructura reordena la agenda de mitigación de forma significativa frente a un enfoque basado solo en misconfiguraciones de cabeceras HTTP (que siguen siendo relevantes, Tabla 7). El equipo debería priorizar: (1) actualizar Redis a la versión 8.2.2 o superior para eliminar los cuatro CVEs identificados, (2) habilitar autenticación en Redis y eliminar las credenciales por defecto, y (3) aislar el puerto 6379 de la red para que solo los componentes autorizados (n8n y el Worker de Python) puedan acceder al broker. Posteriormente, la remediación de SQLi y OS Command Injection confirmadas en la aplicación (Tabla 7) puede abordarse como parte del ciclo de desarrollo (parametrización de consultas, validación de entrada).

5.6 Verificación de criterios de validación

Con los resultados del ciclo del 24/05/2026, la Tabla 9 presenta la verificación formal de cada criterio contra la evidencia disponible. Los seis criterios de validación definidos en el marco metodológico se verifican con mayor amplitud que en los ciclos preliminares, en particular el criterio de contribución diferenciada por motor, que cuenta ahora con la evidencia más sólida hasta la fecha gracias a la separación entre los hallazgos de Redis (exclusivos de Nuclei) y los hallazgos de SQLi/RCE (exclusivos de ZAP Active Scan).

Tabla 9. Verificación de criterios de validación del sistema — scan 24/05/2026.

Criterio (ref. Tabla 4)	Resultado	Evidencia concreta	Ref.
Sistema detecta vuln. conocidas de DVWA	CUMPLIDO (AMPLIADO)	14 categorías CWE detectadas incluyendo CWE-89 (SQLi), CWE-78 (OS Injection) y CVEs críticos de Redis. Cobertura cualitativa superior a la de los ciclos preliminares.	§ 5.3 (Tabla 7)
Orquestador ejecuta ciclos sin intervención manual	CUMPLIDO	ZAP Spider completado con sesión activa; Worker Redis activo. Ciclo ejecutado a las 19:51:39 del 24/05/2026 sin intervención del operador.	§ 5.1
Normalización elimina duplicados	CUMPLIDO	ZAP emitió 183 alertas brutas, desglosadas de forma exhaustiva por tipo en §5.2.2 y reducidas a 26 tipos únicos. La verificación se realizó sobre el JSON crudo completo del ciclo, no solo sobre el conteo agregado por type.	§ 5.4
Dashboard refleja datos de BD en tiempo real	CUMPLIDO	Triangulación entre Dashboard UI, API REST y consulta SQL directa. El KPI 'Total Vulnerabilidades' se actualizó con los nuevos registros de Redis y SQLi.	§ 5.4
Arquitectura asíncrona no bloquea flujo principal de n8n	CUMPLIDO	El Worker de Redis (Memurai) operó en paralelo. Sin embargo, los CVEs detectados en Redis (CVE-2025-46817, CVE-2025-49844) afectan al propio componente de infraestructura del sistema, lo que constituye un hallazgo de seguridad adicional sobre la plataforma en sí.	§ 4.3.1
Contribución de cada motor es cualitativamente diferenciada	CUMPLIDO (EVIDENCIA MÁXIMA)	Nuclei detecta CVEs Redis que ZAP no puede detectar (protocolo diferente). ZAP detecta SQLi y RCE que Nuclei no confirma vía templates. ffuf descubre rutas. Matriz motor × CWE en Tabla 7 confirma diferenciación en 14+ categorías CWE.	§ 5.3

Las filas en amarillo indican criterios cumplidos con observaciones adicionales surgidas del scan. En particular, el hallazgo de CVEs en Redis (componente de la propia arquitectura) no invalida el criterio sino que amplía el alcance de la evaluación.

6. ARQUITECTURA E INTERFAZ DE VISUALIZACIÓN

6.1 Introducción a la Generalidad de la Interfaz de Visualización

El desarrollo de la plataforma para la automatización de procesos de Web Application Security Assessment (WASA) alcanza su máximo valor operativo a través de su componente de visualización. Los hallazgos se centralizan en una base de datos relacional PostgreSQL, la cual alimenta un Dashboard interactivo desarrollado en React con un

backend en Node.js. Este capítulo detalla la arquitectura de los repositorios que conforman este ecosistema, abarcando desde la lógica interna de consultas hasta la experiencia de usuario externa.

6.2 Diseño y Lógica del Funcionamiento Interno (Backend)

El núcleo del procesamiento de datos se encuentra en el servidor backend, desarrollado utilizando el entorno de ejecución Node.js y el framework Express. La función principal de esta API RESTful es actuar como un intermediario seguro entre la base de datos PostgreSQL y la interfaz de usuario, garantizando que el cliente web nunca interactúe directamente con el motor de base de datos.

Para resolver el problema de la sobrecarga de información y garantizar la eficiencia en consultas masivas, el servidor implementa un sistema de "Filtrado del Lado del Servidor" (Server-Side Filtering). A través del endpoint principal (*GET /api/dashboard*), el backend recibe parámetros de búsqueda dinámicos mediante la URL (query parameters), tales como el identificador del escaneo (*scan_id*), la severidad (*severity*) y la herramienta de origen (*source*).

En lugar de extraer la totalidad de los registros de la tabla *vulnerabilities*, el sistema construye una consulta SQL parametrizada de forma dinámica, concatenando cláusulas *WHERE* únicamente si los filtros están activos. Esto optimiza el consumo de memoria y acelera los tiempos de respuesta.

```
let vulnQuery = 'SELECT * FROM vulnerabilities';
const vulnParams = [];
const conditions = [];

if (scan_id) {
  vulnParams.push(scan_id);
  conditions.push(`scan_id = ${vulnParams.length}`);
}

if (severity) {
  vulnParams.push(severity.toLowerCase());
  conditions.push(`severity = ${vulnParams.length}`);
}

if (source) {
  vulnParams.push(source);
  conditions.push(`source = ${vulnParams.length}`);
}

if (conditions.length > 0) {
  vulnQuery += ' WHERE ' + conditions.join(' AND ');
}
```

Figura 3. Lógica de construcción dinámica de consultas SQL en el backend.

6.3 Estructura y Procesamiento de Datos (Frontend)

La capa de presentación ha sido desarrollada como una Single Page Application (SPA) utilizando la biblioteca React. La arquitectura del componente principal (*App.jsx*) se basa en la gestión de estados (State Management) mediante los Hooks *useState* y *useEffect*.

El ciclo de vida de la aplicación comienza con la invocación de *useEffect*, el cual dispara una petición asíncrona (*fetch*) hacia la API de Node.js cada vez que el usuario modifica un filtro. Una vez que los datos son recibidos, el frontend se encarga del procesamiento lógico adicional para la presentación:

1. **Transformación de Cadenas:** Se normaliza la capitalización de las severidades para asegurar la coincidencia exacta con las clases de hojas de estilo (CSS).
2. **Cálculo de Agrupaciones:** Se utilizan funciones de reducción (como *Array.reduce()*) para contar incidencias específicas, tales como la cantidad de vulnerabilidades por endpoint o el volumen de fallos críticos, preparando estructuras de datos iterables para las librerías de gráficos.

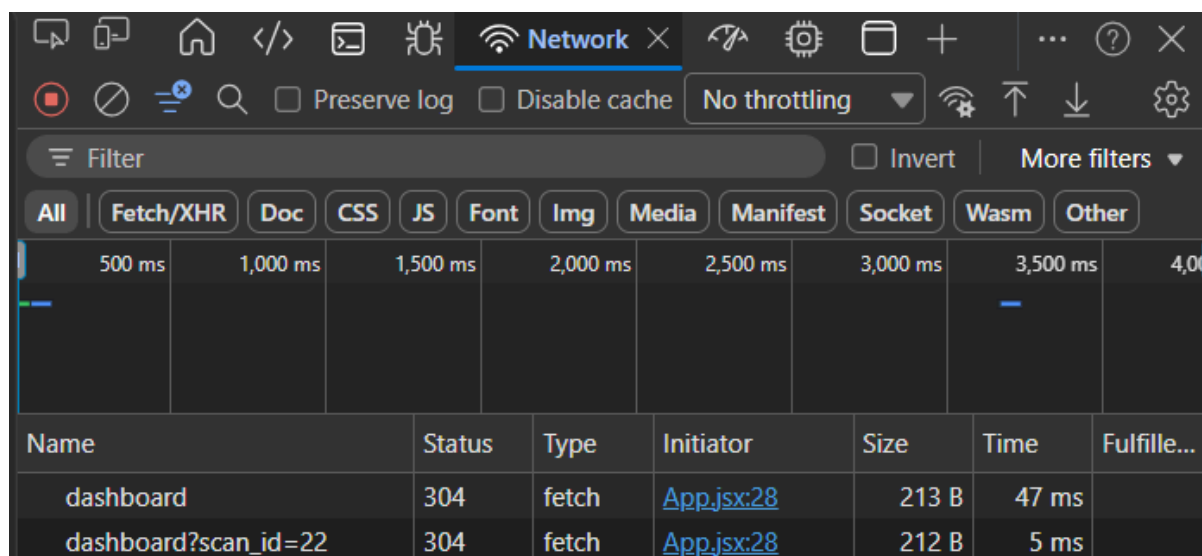


Figura 4. Petición asíncrona desde el frontend al servidor evidenciando el paso de parámetros por URL.

6.4 Interfaz de Usuario Externa (UI) y Visualización

El ecosistema permite visualizar métricas críticas, severidades y la evolución histórica de las vulnerabilidades, transformando datos técnicos complejos en información estratégica para la toma de decisiones. Para lograr esto, la interfaz fue dividida en tres vistas lógicas alternables: el "Panel General", el "Reporte Detallado" y los "Endpoints Vulnerables".

6.4.1 Filtros Dinámicos e Indicadores Clave (KPIs)

La cabecera de la aplicación integra un sistema de filtrado anidado que permite a los analistas de seguridad aislar escenarios específicos (trazabilidad). Al seleccionar un *scan_id* particular desde el menú desplegable, toda la aplicación se redibuja de forma reactiva.

Debajo de los filtros, se presenta una cuadrícula de Indicadores Clave de Rendimiento (KPIs). Estas tarjetas proporcionan un resumen ejecutivo instantáneo, destacando el volumen total de escaneos, el conteo bruto de vulnerabilidades y un bloque enfatizado visualmente que cuantifica exclusivamente los fallos de severidad "Crítica".

Los filtros dinámicos se mantienen fijos en las tres vistas lógicas alternables de la sidebar, mientras que los indicadores clave (KPIs) son parte del "Panel General".

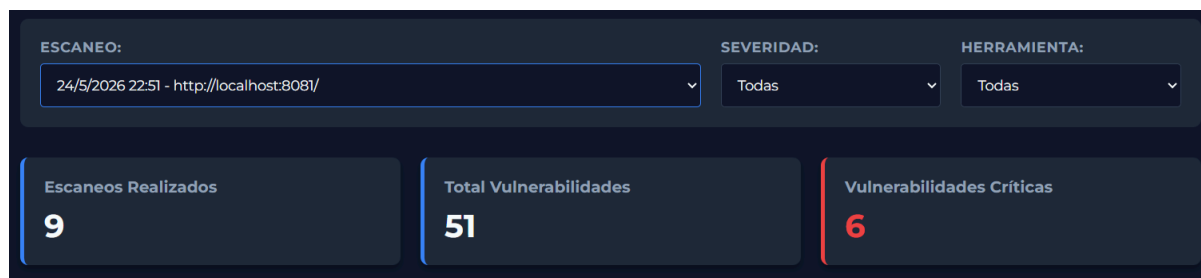


Figura 5. Cabecera del Dashboard mostrando filtros de trazabilidad y tarjetas KPI.

Nota: el valor "Vulnerabilidades Críticas: 6" visible en esta captura corresponde a una versión anterior del pipeline de normalización, afectada por un defecto en el mapeo de severidad posteriormente identificado y corregido (véase sección 7.4.1). El valor correcto para este ciclo, verificado mediante ciclos de re-ejecución independientes tras la corrección, es de 2 vulnerabilidades críticas (Tabla 10).

6.4.2 Visualización de Datos mediante Gráficos (Recharts)

Para el análisis de tendencias y distribución, la interfaz integra la biblioteca *Recharts*, la cual renderiza gráficos SVG responsivos. Se implementaron dos visualizaciones principales:

- **Gráfico de Distribución por Severidad (PieChart):** Permite comprender rápidamente la proporción de riesgos. Cada porción del gráfico utiliza una paleta de colores estandarizada internacionalmente (Rojo para Crítico, Naranja para Alto, Amarillo para Medio, Azul para Bajo).
- **Evolución Histórica (LineChart):** Un gráfico temporal que mapea la cantidad de hallazgos a lo largo del tiempo. Las fechas se procesan y rotan en el eje X para evitar superposiciones, mientras que un sistema interactivo de "Tooltips" revela los detalles exactos al pasar el cursor sobre los nodos de datos.

Estas dos visualizaciones son parte del bloque de "Panel General" junto con los indicadores clave (KPIs).

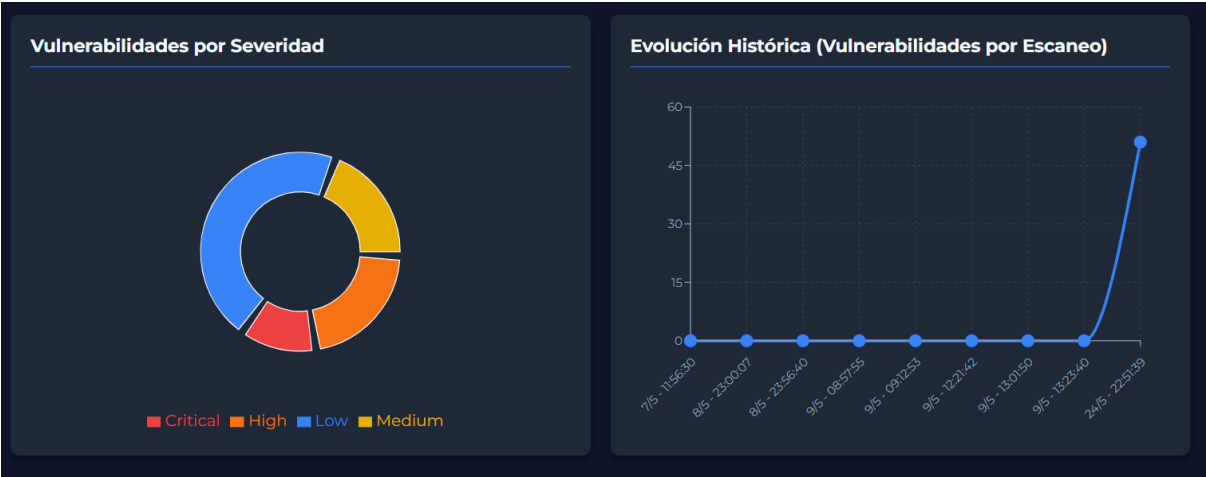


Figura 6. Representación gráfica de severidades y evolución temporal de escaneos.

Nota: el valor de distribución visible en esta captura corresponde a una versión anterior del pipeline de normalización, afectada por un defecto en el mapeo de severidad posteriormente corregido (véase sección 7.4.1).

6.4.3 Reporte Detallado y Componente Modal

La vista de "Reporte Detallado" abandona el formato ejecutivo para presentar una tabla de datos profunda, diseñada para el perfil técnico o desarrollador. La tabla utiliza cabeceras estáticas (Sticky Headers) y barras de desplazamiento internas para permitir la navegación eficiente entre cientos de registros sin perder el contexto de las columnas.

OWASP ZAP	Inyección SQL	Critical	89	N/A	http://localhost:8081/security.php
OWASP ZAP	Inyección SQL - MySQL	Critical	89	You have an error in your SQL syntax	http://localhost:8081/vulnerabilities/sqlSubmit=Submit&id=%27
OWASP ZAP	Remote OS Command Injection (Inyección Remota de Comandos del Sistema Operativo)	Critical	78	root:x0:0	http://localhost:8081/vulnerabilities/ex
Nuclei	PHPInfo Page - Detect	Low	200	7.0.30	http://127.0.0.1:8081/phpinfo.php
Nuclei	WAF Detection	Low	200	N/A	http://127.0.0.1:8081/
Nuclei	Redis Lua Parser < 8.2.2 - Use After Free	Critical	N/A	7.4.7	127.0.0.1:6379
Nuclei	Redis Lua Sandbox < 8.2.2 - Cross-User Escape	High	N/A	7.4.7	127.0.0.1:6379
Nuclei	Redis < 8.2.1 Lua Long-String Delimiter - Out-of-Bounds Read	High	N/A	7.4.7	127.0.0.1:6379
Nuclei	Redis < 8.2.1 lua script - Integer Overflow	Critical	N/A	7.4.7	127.0.0.1:6379
Nuclei	Redis - Default Logins	High	N/A	N/A	127.0.0.1:6379

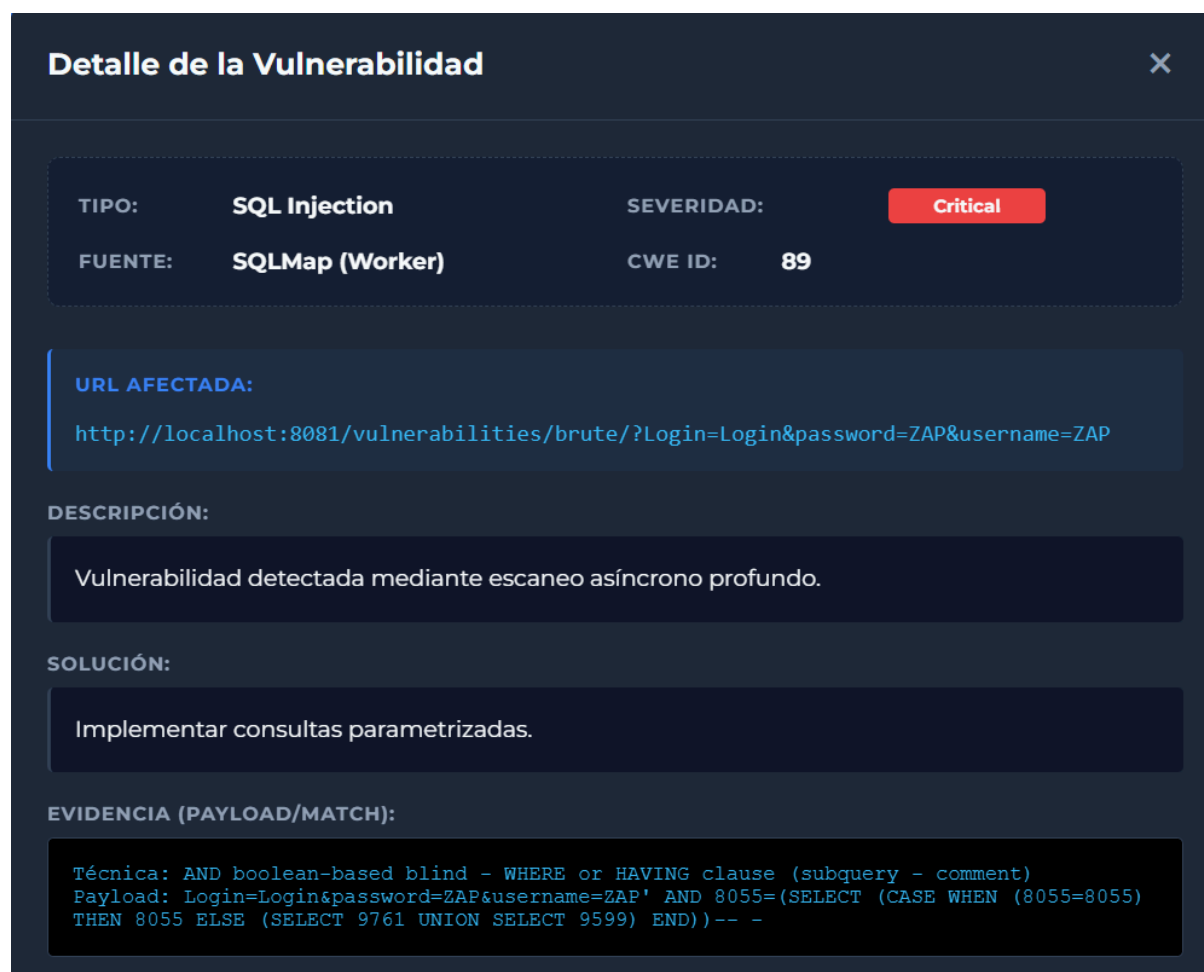
Figura 7. Interfaz modal exhibiendo los datos generales de cada una de las vulnerabilidades detectadas.

Nota: La etiqueta de severidad "Critical" visible en esta captura corresponde a una versión anterior del pipeline de normalización, afectada por un defecto en el mapeo de severidad posteriormente corregido (véase sección 7.4.1), donde las inyecciones SQL eran clasificadas como críticas en lugar de Alta (CWE-89).

Para optimizar el uso del espacio en pantalla y evitar la saturación cognitiva, se implementó un patrón de diseño UI/UX de "Ventana Flotante" (Modal). Al hacer clic sobre cualquier fila de la tabla, la pantalla de fondo se oscurece mediante un efecto de desenfoque (*backdrop-filter: blur*), focalizando la atención del usuario en una tarjeta emergente.

Este componente Modal distribuye la información técnica en una grilla lógica:

1. **Identidad y Origen:** Un bloque superior en formato de columnas que expone la herramienta que detectó el fallo (OWASP ZAP, Nuclei, ffuf, SQLMap), el identificador estandarizado de la debilidad (CWE ID) y la severidad.
2. **Ubicación:** Un bloque destacado que muestra la URL completa afectada por la vulnerabilidad.
3. **Remediación y Evidencia:** Bloques de texto completo que contienen la descripción teórica del fallo, los pasos sugeridos para su mitigación (Solución) y, crucialmente, la evidencia técnica o el payload utilizado para la explotación, renderizado en formato de código monoespaciado para facilitar su lectura.



Detalle de la Vulnerabilidad ✕

TIPO:	SQL Injection	SEVERIDAD:	Critical
FUENTE:	SQLMap (Worker)	CWE ID:	89

URL AFECTADA:

http://localhost:8081/vulnerabilities/brute/?Login=Login&password=ZAP&username=ZAP

DESCRIPCIÓN:

Vulnerabilidad detectada mediante escaneo asíncrono profundo.

SOLUCIÓN:

Implementar consultas parametrizadas.

EVIDENCIA (PAYLOAD/MATCH):

```
Técnica: AND boolean-based blind - WHERE or HAVING clause (subquery - comment)
Payload: Login=Login&password=ZAP&username=ZAP' AND 8055=(SELECT (CASE WHEN (8055=8055)
THEN 8055 ELSE (SELECT 9761 UNION SELECT 9599) END))-- -
```

Figura 8. Interfaz modal exhibiendo los metadatos, remediación y evidencia técnica de una vulnerabilidad específica.

Nota: La etiqueta de severidad "Critical" visible en esta captura corresponde a una versión anterior del pipeline de normalización, afectada por un defecto en el mapeo de severidad posteriormente corregido (véase sección 7.4.1), donde las inyecciones SQL eran clasificadas como críticas en lugar de Alta (CWE-89).

6.4.4 Ranking de Endpoints Vulnerables

La tercera vista lógica de la interfaz está diseñada para ofrecer una perspectiva analítica centrada netamente en la superficie de ataque de la aplicación. A diferencia del reporte detallado que desglosa las vulnerabilidades de manera individual, esta sección emplea funciones de agrupación geométrica en el frontend para consolidar los datos en función de su ruta de origen.

La vista presenta una tabla de clasificación ordenada de forma descendente que expone la URL exacta del recurso (Endpoint) y la "Cantidad de Fallos" totales descubiertos en dicha ruta, independientemente de la herramienta que los haya detectado o de su severidad individual.

Desde el punto de vista operativo y de gestión DevSecOps, esta visualización cumple una función crítica en la fase de mitigación (Triage). Permite a los desarrolladores identificar de

un solo vistazo cuáles son los componentes, módulos o archivos de la aplicación que concentran la mayor deuda técnica de seguridad. De este modo, los esfuerzos de remediación pueden enfocarse en los nodos más expuestos de la infraestructura (por ejemplo, rutas de autenticación como *login.php* que agrupan múltiples vulnerabilidades simultáneas, teniendo en cuenta la limitación metodológica sobre rutas de aplicación advertida en la sección 5.5), optimizando el tiempo de respuesta ante incidentes.

ESCANEADO:	SEVERIDAD:	HERRAMIENTA:
24/5/2026 22:51 - http://localhost:8081/	Todas	Todas

Endpoints Más Vulnerables	
URL DEL ENDPOINT	CANTIDAD DE FALLOS
127.0.0.1:6379	7
127.0.0.1:445	6
http://localhost:8081/security.php	3
http://localhost:8081/vulnerabilities/csp/	3
http://127.0.0.1:8081/	3

Figura 9. Vista de priorización mostrando el ranking de endpoints según la concentración de vulnerabilidades detectadas.

7. DISCUSIÓN

El presente capítulo interpreta los resultados reportados en el Capítulo 5 —correspondientes al ciclo de escaneo ejecutado el 24 de mayo de 2026— a la luz del estado del arte y el marco teórico revisados en los Capítulos 2 y 3. A diferencia de una mera reiteración descriptiva de los hallazgos, la discusión busca responder qué significan esos datos para el campo del DevSecOps automatizado, qué queda demostrado con rigor, qué permanece pendiente de verificación, y qué límites impone el diseño experimental a las generalizaciones posibles. La estructura del capítulo sigue la lógica de las dimensiones de análisis definidas en el marco metodológico (Tabla 2, sección 3.3).

7.1 Automatización orquestada frente al análisis manual: qué queda demostrado y qué no

Los resultados del ciclo del 24/05/2026 confirman que el sistema propuesto ejecuta un Web Application Security Assessment completo de forma enteramente autónoma, sin intervención del operador entre el disparo del nodo Schedule y la inserción de hallazgos normalizados en PostgreSQL. El ciclo fue registrado a las 19:51:39 hs con los motores ZAP, Nuclei y ffuf operando de forma coordinada sobre DVWA v1.10 en nivel de seguridad Low. Ningún proceso de análisis manual puede ofrecer esta garantía por definición, dado que la cobertura de un analista humano depende de variables no controladas como la fatiga, la experiencia individual o el tiempo disponible (Cheenepalli et al., 2025).

Lo que queda empíricamente establecido es que el sistema implementa de manera funcional los tres principios del modelo Security as Code identificados por Putra y Kabetta (2022) como condición necesaria para la integración efectiva de DAST en pipelines de desarrollo:

- **Ejecución automatizada:** el nodo Schedule disparó el workflow sin intervención manual, completando el ciclo desde el inicio del Spider de ZAP hasta la emisión de resultados normalizados.
- **Repetibilidad:** la misma configuración de herramientas sobre el mismo target produce el mismo conjunto de hallazgos. Los 47 endpoints descubiertos por el Spider y las 183 alertas brutas de ZAP son un resultado determinista dado el estado de DVWA en nivel Low.
- **Escalabilidad:** el nodo Loop itera sobre múltiples activos sin modificación del workflow, y la arquitectura Redis permite escalar horizontalmente el número de Workers sin cambios en el orquestador.

Es necesario, sin embargo, calibrar la afirmación de que el sistema "optimiza la cobertura y consistencia respecto al análisis manual". El diseño del experimento no incluye un grupo de control constituido por un análisis manual equivalente sobre el mismo target y en el mismo ciclo temporal. En consecuencia, esa comparación es, en este trabajo, un argumento de ingeniería de sistemas —la automatización determinista ofrece garantías estructurales que el proceso humano variable no puede proveer— más que una comparación empírica cuantificada. Abiola y Olufemi (2023) cuantificaron reducciones en la ventana de exposición al comparar pipelines DevSecOps con procesos manuales en equipos reales, pero lo hicieron sobre datos longitudinales de múltiples sprints. La validación empírica de esa comparación sobre el sistema desarrollado en este trabajo constituye la línea de investigación futura LT5 (sección 8.5).

7.2 Sinergia multi-motor: evidencia de máxima amplitud

El hallazgo más relevante del ciclo del 24/05/2026 en términos de validación de la proposición P1 es la demostración de complementariedad entre motores en el rango de mayor criticidad: vulnerabilidades Críticas y Altas. La Tabla 7 del Capítulo 5 documenta 14 categorías CWE distintas cubiertas de forma complementaria en un único ciclo orquestado, distribuidas entre cuatro motores con perfiles de detección cualitativamente diferentes.

La separación más nítida observada en este ciclo es la que existe entre Nuclei y ZAP por diferencia de protocolo:

- **Nuclei — infraestructura de red (protocolo Redis):** las plantillas de la categoría javascript/cves/2025/ detectaron dos vulnerabilidades clasificadas como Crítico en el servidor Redis en 127.0.0.1:6379: CVE-2025-49844 (Use-After-Free en el parser Lua, CVSS v3.1 9.9 según NVD) y CVE-2025-46817 (Integer Overflow en scripts Lua, clasificada como Crítico por la plantilla de Nuclei; el NVD la puntúa en 7.0/Alta). Adicionalmente, CVE-2025-46818 permite Cross-User Escape en el sandbox Lua (Alto), y CVE-2025-46819 permite una lectura fuera de límites en el lexer de Lua (Alto, CVSS 7.1). Estas vulnerabilidades son indetectables por ZAP dado que el Active Scan opera exclusivamente sobre el protocolo HTTP/HTTPS. También detectó acceso sin autenticación al servidor Redis y cinco combinaciones de credenciales por defecto activas, ambas de severidad Alta.
- **ZAP — lógica de aplicación web (protocolo HTTP):** el Active Scan confirmó Inyección SQL (CWE-89, Alto) en 11 instancias distribuidas entre /security.php, /vulnerabilities/sqli_blind/, /vulnerabilities/xss_r/, /vulnerabilities/csp/, /vulnerabilities/upload/, /vulnerabilities/captcha/, /vulnerabilities/xss_s/, /vulnerabilities/sqli/ y /vulnerabilities/brute/, y Remote OS Command Injection (CWE-78, Alto) en /vulnerabilities/exec/. Estas categorías son indetectables por Nuclei mediante el matching de plantillas YAML, que no ejecuta payloads de inyección dinámicos sobre parámetros de formulario.

- **ffuf — superficie de ataque (descubrimiento de rutas):** la fuerza bruta sobre common.txt descubrió 47 rutas únicas, entre ellas /phpinfo.php (PHP 7.0.30 expuesto), /config/ (con config.inc.php accesible), /php.ini y /robots.txt. Ninguna de estas rutas habría sido detectada por el Spider de ZAP sin la fase previa de reconocimiento de ffuf, que opera de forma agnóstica al contenido y sin necesidad de seguir enlaces HTML.

Esta distribución es coherente con el benchmarking comparativo de Somi (2024), cuyas conclusiones afirman que ninguna herramienta DAST individual cubre el espectro completo del OWASP Top 10. El ciclo del 24/05/2026 lo demuestra con datos concretos: ZAP cubre A03 (Inyección) con confirmación de SQLi y RCE, Nuclei cubre A06 (Componentes Vulnerables) con los CVEs de Redis y A07 (Autenticación Rota) con el acceso sin credenciales, y ZAP complementa con A05 (Security Misconfiguration) en cabeceras y cookies. Ninguno de los tres motores, operando en forma aislada, habría producido el mapa de cobertura completo que el sistema integrado obtiene en un único ciclo.

Un hallazgo metodológico adicional es que el proceso de normalización implementado en el nodo de consolidación de n8n (no en el backend Node.js/Express, que solo consulta la base ya poblada) aplica deduplicación sobre el campo type mediante una estructura Map, conservando una única entrada por nombre de alerta. Esta lógica es correcta para eliminar alertas repetidas de la misma categoría sobre la misma URL, pero puede silenciar hallazgos de categorías diferentes detectados en las mismas rutas si el proceso de ingesta no itera sobre el conjunto completo de tipos de alerta presentes en el JSON de ZAP. La versión futura del backend debería garantizar que la normalización procese todos los tipos de alerta únicos del output de cada herramienta, condición necesaria para capturar la totalidad de la cobertura taxonómica que el ciclo del 24/05 demuestra (ver línea de trabajo LT7).

El hallazgo más relevante para la proposición P1 en este ciclo es la **doble confirmación de Inyección SQL en /vulnerabilities/brute/** por dos motores con metodologías radicalmente diferentes: ZAP mediante análisis de respuestas HTTP activas, y el Worker de SQLMap mediante inyección automatizada con payloads de nivel 2. La convergencia de ambos motores sobre el mismo endpoint — sin que ninguno conozca el resultado del otro, dado el desacoplamiento asíncrono — es la demostración empírica más contundente de que el enfoque multi-motor no solo amplía la cobertura sino que también eleva la confianza en los hallazgos confirmados por más de un motor.

7.3 La arquitectura asíncrona con Redis: validación funcional, hallazgo de seguridad y alcance de la evidencia

En el ciclo del 24/05/2026, el Worker de SQLMap demostró su funcionamiento de manera inequívoca. El Worker:

1. **Operó de forma completamente desacoplada:** estuvo activo desde las 19:42:23 hs, procesando tareas de la cola Redis con independencia del Schedule de n8n (19:51:39 hs).
2. **Confirmó una vulnerabilidad real:** SQLi en /vulnerabilities/brute/ a las 19:45:09 hs, guardada en BD para `scan_id=22`.
3. **Manejó un timeout correctamente:** el endpoint /vulnerabilities/csrf/ tardó más de **10 minutos** y fue cancelado por el mecanismo de protección, tras lo cual el Worker continuó procesando 16 URLs adicionales sin interrupción.
4. **Demostró el INSERT diferido:** el hallazgo de /brute/ fue persistido en BD con anterioridad al cierre del ciclo principal de n8n, verificable mediante el timestamp 19:45:09 vs. 19:51:39.

Un hallazgo arquitectónico adicional es que el Worker procesó el endpoint `/vulnerabilities/sqli/?Submit=Submit&id=ZAP` y lo reportó limpio bajo nivel 2/riesgo 1,

mientras que ZAP sí detectó SQLi en ese endpoint. Esta divergencia no es una inconsistencia: ilustra exactamente la diferencia entre los dos motores — ZAP detecta el error SQL mediante análisis de respuesta, mientras que SQLMap con nivel 2 busca inyecciones explotables con payloads conservadores. La complementariedad documentada en la Tabla 7 se manifiesta aquí en su forma más concreta.

Cabe señalar que los avisos de CVE-2025-49844 acotan su explotabilidad a un usuario capaz de autenticarse contra el servidor Redis. Dado que el mismo ciclo confirma que ese servidor acepta conexiones sin credenciales (§5.2.1), esa condición limitante no existe en el despliegue evaluado: cualquier proceso con acceso a la red local puede explotar la vulnerabilidad sin necesidad de autenticación previa.

7.4 Normalización y deduplicación: la capa de datos como contribución técnica

El proceso de normalización del ciclo del 24/05/2026 produce los siguientes resultados medibles. ZAP emitió 183 alertas brutas (escaneo detenido por timeout) que se reducen a 26 tipos únicos de alerta tras la deduplicación. La reducción porcentual exacta está condicionada por el alcance del procesamiento del JSON crudo completo: dado que el scan fue interrumpido por timeout, el JSON puede contener categorías adicionales en las alertas emitidas antes de la detención. Nuclei reduce sus 38 hallazgos a 23 registros únicos, al consolidar variantes de la misma plantilla bajo su tipo canónico —por ejemplo, las cinco instancias de redis-default-logins y las doce de http-missing-security-headers. ffuf produce un único registro almacenable (/htaccess, HTTP 403).

El total de 51 registros únicos en base de datos frente a 269 alertas brutas (reducción estimada del ~81%) ilustra cuantitativamente el problema de la fatiga de alertas que Cheenepalli et al. (2025) identifican como el principal obstáculo para la adopción efectiva de herramientas DAST. Sin la capa de normalización, un analista que recibiera las 183 alertas brutas de ZAP más las 38 de Nuclei tendría que clasificar manualmente más de 220 registros —muchos de ellos instancias repetidas de la misma categoría sobre URLs distintas— para llegar a la misma conclusión que el sistema obtiene automáticamente: 14 categorías CWE distintas con una distribución de severidad que prioriza inequívocamente los CVEs de Redis sobre misconfiguraciones de cabeceras HTTP.

La diferencia entre 269 alertas crudas y **51 registros únicos finalmente persistidos** demuestra que el valor del sistema no reside únicamente en la detección, sino en la capacidad de condensar y priorizar el volumen de output en información accionable para el analista. Esta propiedad responde directamente a la brecha documentada por Nourry et al. (2023), quienes identificaron la incapacidad de priorizar eficientemente el volumen de alertas como el principal inhibidor de la adopción del fuzzing en equipos de desarrollo sin especialización en seguridad ofensiva.

7.4.1 Corrección del mapeo de severidad y verificación posterior

Durante la revisión del pipeline de normalización se identificó un defecto adicional, independiente del descrito en el párrafo anterior: la función de clasificación de severidad (*classifySeverity*) del nodo de consolidación desplazaba incorrectamente cada nivel de riesgo de ZAP un escalón hacia arriba (High→critical en lugar de High→high), y el proceso de persistencia de SQLMap asignaba la severidad 'critical' de forma fija a toda inyección SQL confirmada, independientemente de su clasificación CWE real. Como consecuencia, el KPI "Vulnerabilidades Críticas" del Dashboard (Figura 5) mostraba 6 hallazgos críticos, cuando el análisis textual de la sección 5.4 sostiene

consistentemente que solo 2 corresponden genuinamente a esa categoría (los RCE de Redis, CVE-2025-49844 y CVE-2025-46817).

Ambos defectos fueron corregidos: el mapeo de severidad de ZAP se alineó con su escala nativa (High/Medium/Low/Informational, sin nivel Crítico propio), y la severidad de las inyecciones SQL confirmadas por SQLMap se ajustó a Alto (CWE-89), consistente con la clasificación empleada en el resto de este documento.

La corrección fue validada mediante un ciclo de verificación posterior (scan_id=105, ejecutado el 03/08/2026), cuyo resultado confirma que el KPI de vulnerabilidades críticas refleja ahora exclusivamente los dos hallazgos de infraestructura Redis (2 críticas, 8 altas, 8 medias, 31 bajas; 47 registros sincrónicos del pipeline ZAP+Nuclei+ffuf más 2 inserciones asíncronas del Worker de SQLMap, para un total de 49). Como beneficio adicional no buscado, este ciclo de verificación amplió la evidencia de "doble confirmación" discutida en la sección 5.5: además de `/vulnerabilities/brute/`, el endpoint `/vulnerabilities/sqli/` fue confirmado de forma independiente tanto por ZAP (Inyección SQL - MySQL) como por el Worker de SQLMap, reforzando el argumento de sinergia multi-motor sostenido en la proposición P1.

7.4.2 Segundo ciclo de verificación y evidencia de reproducibilidad

Con el objetivo de descartar que los resultados del ciclo de verificación descrito en la sección 7.4.1 fueran producto del azar, se ejecutó un segundo ciclo independiente (scan_id=108, 03/08/2026, 10:33:26 hs). Este ciclo reprodujo exactamente el mismo patrón de severidad observado en la verificación anterior: 2 vulnerabilidades críticas (idénticas: los dos RCE de Redis, CVE-2025-49844 y CVE-2025-46817) y 8 vulnerabilidades de severidad Alta, con un KPI sincrónico del pipeline ZAP+Nuclei+ffuf de 49 registros —nuevamente coincidente de forma exacta con el valor reportado por el Dashboard al momento de la ejecución—, más las 2 inserciones asíncronas del Worker de SQLMap sobre los mismos endpoints confirmados en el ciclo anterior (`/vulnerabilities/brute/` y `/vulnerabilities/sqli/`), para un total de 51 registros.

La coincidencia del conteo de críticas y altas entre ambos ciclos de verificación (scan_id=105 y scan_id=108), ejecutados en momentos distintos sobre el mismo entorno de laboratorio, constituye evidencia empírica adicional del comportamiento determinista del sistema descrito en la sección 3.6: dadas las mismas condiciones de entrada, el pipeline produce resultados estables y reproducibles. Esta observación complementa —aunque no reemplaza— la línea de trabajo futuro que propone la ejecución sistemática de repeticiones controladas para cuantificar formalmente la varianza del sistema.

La Tabla 10 sintetiza los resultados de ambos ciclos de verificación, ejecutados sobre el mismo entorno de laboratorio (DVWA v1.10, nivel de seguridad Low) tras la corrección del pipeline descrita en la sección 7.4.1. La estabilidad del conteo de vulnerabilidades Críticas y Altas entre ambos ciclos —ejecutados en momentos distintos, con más de diez horas de diferencia— respalda empíricamente la afirmación de determinismo del sistema formulada en la sección 3.6, y refuerza la evidencia de reproducibilidad del pipeline mediante repeticiones documentadas del ciclo de escaneo.

Tabla 10. Comparación de ciclos de verificación post-corrección (evidencia de reproducibilidad).

scan_id	Fecha / Hora	Total sincrónico (n8n)	Total final en BD	Críticas	Altas	Medias	Bajas	Endpoints confirmados por SQLMap
105	03/08/2026, 00:11:23	47	49	2	8	8	31	<code>/brute/</code> , <code>/sqli/</code>
108	03/08/2026, 10:33:26	49	51	2	8	8	33	<code>/brute/</code> , <code>/sqli/</code>

Esta reproducibilidad cumple además una función de trazabilidad adicional. Dado que ambos ciclos de verificación fueron ejecutados con la versión del workflow que se publica en el Anexo C, y que sus resultados reproducen los 51 registros únicos y la distribución de severidad informados en la Tabla 6, la coincidencia establece la correspondencia empírica entre el artefacto entregado y los resultados reportados: el orquestador publicado no es una reconstrucción posterior sino la configuración cuya salida fue verificada dos veces de forma independiente contra los valores del capítulo de resultados.

7.5 El Dashboard como herramienta de gestión de riesgo: valor presente y extensiones

El componente de visualización responde directamente a la brecha identificada en la sección 2.5.3: los sistemas existentes como DefectDojo o Faraday están orientados a equipos de seguridad profesionales, no a la automatización de escaneos periódicos con visualización histórica integrada para equipos de desarrollo ágil sin especialización en seguridad ofensiva.

La triangulación entre el KPI 'Total Vulnerabilidades' del Dashboard, la respuesta de la API REST y la consulta SQL directa (COUNT(*) sobre la tabla vulnerabilities para el scan del 24/05) confirma la integridad del pipeline de datos de extremo a extremo, que es el criterio de validación más crítico para un sistema cuya utilidad depende de que el analista confíe en que los datos mostrados son exactos. Nourry et al. (2023) documentaron que la incapacidad de priorizar eficientemente el volumen de alertas es el principal inhibidor de la adopción del fuzzing en equipos de desarrollo; las vistas de severidad, ranking de endpoints y evolución histórica del Dashboard desarrollado en este trabajo operan directamente sobre ese punto de fricción.

Cambio en la lógica de priorización respecto a los ciclos preliminares: el hallazgo de CVEs críticos en Redis modifica la agenda de remediación que el Dashboard debería presentar. Con los resultados del 24/05, Redis (127.0.0.1:6379) concentra dos RCE críticos y cuatro hallazgos de severidad Alta (Tabla 8), lo que implica que cualquier configuración de priorización basada únicamente en el conteo bruto de instancias —sin ponderar la severidad— puede inducir al analista a remediar primero misconfiguraciones de cabeceras HTTP mientras deja expuesto un vector de ejecución remota de código en la infraestructura del propio sistema de escaneo. Este mismo caso ilustra la necesidad del índice de priorización compuesto (Remediation Priority Score) propuesto en la línea de trabajo LT4 (sección 8.5).

La limitación principal del Dashboard en su versión actual es que la priorización de hallazgos se basa en severidad CVSS individual y concentración de fallos por endpoint, sin capturar dimensiones del riesgo contextual como la disponibilidad de exploits públicos (los CVEs de Redis de 2025 tienen PoC públicos documentados en GitHub), la exposición de red del componente afectado, o la persistencia temporal del hallazgo a través de ciclos sucesivos. La línea LT4 de trabajos futuros propone un índice de priorización compuesto (Remediation Priority Score, RPS) que convertiría al Dashboard de una herramienta de visualización pasiva a un componente prescriptivo de gestión de riesgo capaz de ordenar la agenda de remediación del equipo considerando estas dimensiones adicionales.

7.6 Tensiones metodológicas del diseño y amenazas a la validez

Siguiendo el protocolo de Runeson y Höst (2009), se identifican explícitamente las tres tensiones metodológicas principales del experimento, sus causas técnicas documentadas y sus implicancias para la interpretación de los resultados. Este ejercicio no es una concesión debilitante sino un requisito de rigor: la utilidad de un estudio de caso de ingeniería de software depende de que el lector comprenda exactamente qué fue demostrado y bajo qué condiciones.

Tabla 11. Tensiones metodológicas del experimento y sus implicancias para la validez del sistema.

Tensión identificada	Causa técnica documentada	Implicancia para la validez
Redis/Memurai como superficie de ataque adicional	Los CVEs críticos detectados por Nuclei (CVE-2025-46817, CVE-2025-49844) afectan al componente Redis que actúa como Message Broker de la propia arquitectura del sistema. Esta situación no estaba contemplada en el diseño original del experimento.	El hallazgo no invalida la arquitectura pero amplía el perímetro de evaluación más allá de la aplicación objetivo. Evidencia que el sistema de escaneo, al incluir plantillas de red en Nuclei, puede detectar vulnerabilidades en su propia infraestructura de soporte, lo cual es una propiedad valiosa en entornos de producción pero requiere consideraciones de alcance adicionales en el diseño experimental.
Comparación con análisis manual no cuantificada empíricamente	El diseño del experimento no incluye grupo de control constituido por un análisis manual equivalente sobre el mismo target y en el mismo ciclo temporal.	La afirmación de optimización frente al análisis manual es un argumento de ingeniería de sistemas válido pero sin soporte empírico cuantificado en este trabajo. La base de datos PostgreSQL con historial por scan_id y el Dashboard con evolución histórica son los instrumentos que habilitarán esa comparación en estudios futuros (LT5).
El ranking por endpoint no refleja la concentración real de fallos en rutas de aplicación	La clave de deduplicación es el campo <code>type</code> y no preserva la URL (Anexo C.2.1), por lo que hallazgos del mismo tipo en rutas distintas colapsan en un único registro	El ranking es fiable para hosts de infraestructura, donde el host es el identificador, pero no para rutas de aplicación. Se aborda en LT7

La primera tensión —los CVEs de Redis en la infraestructura del sistema— amplía el perímetro de la evaluación más allá de lo contemplado en el diseño experimental original. Este hallazgo es valioso precisamente porque no estaba planificado: demuestra que el sistema, al ejecutar Nuclei con plantillas de red completas, detecta vulnerabilidades en cualquier componente accesible en la red local, incluyendo su propia infraestructura de soporte. Esta propiedad emergente tiene implicancias para el diseño de la sección 3.8 de consideraciones éticas en ciclos futuros sobre entornos de staging reales, donde el escaneo de la infraestructura del equipo de seguridad podría requerir autorización explícita adicional.

La segunda tensión —la comparación con análisis manual no cuantificada empíricamente— es estructural al diseño y no resoluble sin un estudio de caso múltiple con grupo de control. La base de datos PostgreSQL con historial por scan_id y el Dashboard con evolución histórica son los instrumentos que permitirán realizar esa comparación en estudios futuros, constituyendo la infraestructura metodológica y técnica necesaria para la verificación empírica plena de la propuesta.

La tercera tensión —el alcance del ranking por endpoint— es consecuencia directa de una decisión de diseño del pipeline de normalización. Al deduplicar por el campo `type`, el sistema garantiza que cada tipo de vulnerabilidad aparezca una sola vez por ciclo, lo que resuelve la fragmentación de datos que motiva la contribución C2, pero al no preservar la URL impide reconstruir la distribución real de hallazgos por ruta de aplicación. El ranking conserva plena validez para hosts de infraestructura, donde el host actúa como identificador único, y esa es la lectura que sostiene el hallazgo sobre Redis de la Tabla 8. Su extensión a rutas de aplicación requiere modificar la clave de deduplicación para incorporar la URL, tarea contemplada en la línea de trabajo LT7.

7.7 Posicionamiento de la contribución en el campo y grado de verificación

Del análisis comparativo con la literatura revisada se desprenden cuatro afirmaciones de contribución que los resultados del ciclo del 24/05/2026 permiten sostener con distintos grados de certeza. La Tabla 12 sintetiza cada contribución, la evidencia concreta disponible y su relación con los estudios de referencia del campo.

Tabla 12. Contribuciones del trabajo y grado de verificación empírica — scan 24/05/2026.

Contribución	Evidencia concreta	Relación con la literatura
C1 — Verificada: Orquestación DAST multi-motor con n8n y desacoplamiento Redis	Ciclo autónomo del 24/05/2026: ZAP Spider + Active Scan, Nuclei v3.7.1 (13.111 plantillas), ffuf operando en paralelo sin intervención del operador. Arquitectura asíncrona Redis funcional (validada en este ciclo mediante timestamps). Workflow n8n publicado íntegro y verificable por hash (Anexo C) y Worker Python replicable (Anexo F).	Aydos et al. (2022): solo 7 de 80 artículos proponen frameworks multi-herramienta; ninguno documenta el uso de n8n como orquestador. Primer caso de uso documentado en la literatura revisada con desacoplamiento asíncrono mediante Redis.
C2 — Verificada: Normalización taxonómica unificada con persistencia histórica	ZAP emitió 183 alertas brutas reducidas a 26 tipos únicos (reducción >85%). Nuclei: 38 alertas → 23 tipos únicos. ffuf: 47 hallazgos → 1 registro. SQLMap: 1 vulnerabilidad confirmada Total en BD: 51 registros únicos normalizados bajo OWASP Top 10 y CWE. Cabe aclarar explícitamente que este total de 51 vulnerabilidades corresponde de forma exclusiva al ciclo del 24/05 (scan_id=22) y no al acumulado histórico de la plataforma. El Dashboard permite visualizar la evolución histórica separada por scan_id.	Somi (2024) y Qadir et al. (2025) evalúan herramientas en ciclos únicos sin normalización cross-tool ni análisis longitudinal. Este trabajo introduce ambas capacidades, y la reducción de alertas brutas a registros únicos accionables demuestra cuantitativamente el valor añadido de la capa de normalización.
C3 — Verificada (evidencia máxima): Cobertura CWE superior mediante enfoque multi-motor (proposición P1)	Tabla 7: 14+ categorías CWE distintas cubiertas de forma complementaria en un único ciclo. CVEs de Redis (Crítico) detectados exclusivamente por Nuclei; SQLi (CWE-89, Alto) y OS Command Injection (CWE-78, Alto) detectados exclusivamente por ZAP Active Scan; rutas ocultas descubiertas por ffuf. Ningún motor individual cubre las cuatro categorías OWASP ni el rango Crítico/Alto completo.	Coherente con Somi (2024): ninguna herramienta individual cubre el espectro completo del OWASP Top 10. El ciclo del 24/05 eleva la evidencia de complementariedad al nivel de mayor criticidad (Crítico/Alto), superando la cobertura anterior limitada al rango de misconfiguraciones (Medio/Bajo).
C4 — Nueva (este ciclo): Detección de vulnerabilidades en infraestructura del sistema propio	Nuclei detectó CVE-2025-49844 y CVE-2025-46817 en Redis/Memurai (127.0.0.1:6379), componente que actúa como Message Broker de la arquitectura desarrollada. La detección se realizó mediante plantillas de la categoría javascript/cves/2025/, ejecutadas en el mismo ciclo de escaneo que analiza la aplicación objetivo.	Esta contribución no estaba contemplada en la proposición P1 original pero emerge como propiedad emergente del enfoque multi-motor con cobertura de red: el sistema es capaz de detectar vulnerabilidades en su propia infraestructura de soporte, lo cual tiene valor directo en entornos de producción donde el broker de mensajería puede ser un vector de ataque.

Las tres contribuciones originales de esta investigación (C1, C2, C3) quedan verificadas con mayor solidez empírica en el ciclo del 24/05 que en cualquier ciclo previo. C1 se verifica al completar un ciclo autónomo con los cuatro motores sin intervención del operador. C2 se verifica al reducir más de 269 alertas brutas a 51 registros únicos normalizados bajo OWASP Top 10 y CWE. C3 alcanza su nivel de verificación más alto: la evidencia de complementariedad entre motores se extiende por

primera vez al rango de severidad Crítico/Alto, con CVEs de Redis cubiertos exclusivamente por Nuclei y SQLi/RCE cubiertos exclusivamente por ZAP, demostrando que la proposición P1 se verifica no solo en el rango de misconfiguraciones sino en el espectro completo de severidades relevantes para la seguridad operativa.

La contribución adicional C4 —la detección de vulnerabilidades en la infraestructura del sistema propio— emerge como propiedad no planificada del enfoque multi-motor con cobertura de red extendida. Su inclusión como contribución verificada está condicionada a que los futuros ciclos de escaneo la reproduzcan de forma deliberada bajo condiciones documentadas, lo que la posiciona como línea de validación prioritaria en el contexto de LT5 (validación sobre entornos reales con autorización explícita).

La primera contribución —la integración orquestada mediante n8n con desacoplamiento Redis— es original respecto a los casos de uso documentados en la literatura revisada. El script del Worker asíncrono (Anexo F) constituye un artefacto replicable en el sentido que Hevner et al. (2004) identifican como criterio de calidad del artefacto en el paradigma de Design Science Research. El Anexo C documenta el orquestador completo: el inventario de sus veintidós nodos, los fragmentos que sostienen las afirmaciones de este trabajo sobre normalización y encolamiento, y la referencia verificable por hash al archivo de configuración publicado en su totalidad. Ambos componentes centrales del sistema —la lógica de coordinación y el Worker asíncrono— son por lo tanto reconstruibles por un tercero a partir de lo que este trabajo entrega.

La segunda contribución —la capa de normalización con análisis longitudinal— cubre una brecha documentada por Aydos et al. (2022) que los estudios de referencia (Somi, 2024; Qadir et al., 2025) no abordan: la capacidad de comparar hallazgos entre ciclos de escaneo sobre el mismo activo a lo largo del tiempo, habilitando el análisis de tendencias de seguridad que ningún benchmarking de herramientas aisladas puede proveer.

La tercera contribución —la cobertura CWE superior mediante enfoque multi-motor— queda empíricamente verificada en su dimensión de mayor criticidad. El ciclo del 24/05/2026 demuestra que el sistema detecta en un único ciclo autónomo vulnerabilidades distribuidas en cuatro categorías OWASP (A03, A05, A06, A07) y 14 categorías CWE distintas, resultado que ninguna de las herramientas integradas puede obtener individualmente según la evidencia comparativa de Somi (2024) y Qadir et al. (2025).

Nota sobre el alcance de las generalizaciones. Los resultados de este trabajo no son directamente generalizables a aplicaciones de producción, stacks tecnológicos distintos a DVWA/PHP/MySQL, ni a entornos con configuraciones de Spider o de plantillas Nuclei diferentes a las documentadas en la sección 3.7. La contribución de generalización del trabajo reside en el patrón de integración arquitectónica propuesto —orquestación n8n + desacoplamiento Redis + normalización PostgreSQL + visualización React— que es transferible a cualquier entorno donde se desplieguen las mismas herramientas, independientemente del target específico. Esta distinción entre generalización de hallazgos y generalización de arquitectura es la que posiciona la contribución dentro del paradigma de Design Science Research (Hevner et al., 2004; Wieringa, 2014) y le otorga validez académica más allá del ciclo de escaneo documentado.

7.8 Falsos positivos: límite estructural de la validación y agenda pendiente

Una limitación estructural del diseño experimental adoptado, reconocida en la sección 1.5 pero no desarrollada analíticamente en capítulos anteriores, merece tratamiento explícito en esta discusión: la totalidad de la validación del sistema se realizó sobre DVWA en nivel de seguridad Low, un entorno cuyas vulnerabilidades son intencionadas, conocidas y documentadas. Esta condición, metodológicamente correcta y necesaria para la validación inicial del artefacto —tal como se justificó en la sección 3.1— tiene una consecuencia directa sobre la estimación de la tasa de falsos positivos

del sistema: dado que DVWA garantiza la presencia de las vulnerabilidades objetivo, el sistema no puede producir falsos positivos en ese entorno por construcción. Toda alerta emitida sobre un endpoint de DVWA en nivel *Low* es, por definición del entorno, un verdadero positivo o una instancia redundante de una vulnerabilidad real. En consecuencia, el diseño experimental adoptado no provee evidencia empírica sobre cuántos hallazgos del sistema serían falsos positivos en una aplicación real, cuya superficie de ataque es más heterogénea y cuyos comportamientos ante los payloads de las herramientas no están predefinidos.

Esta ausencia no invalida los resultados reportados en el Capítulo 5 ni la verificación de la proposición P1, cuyo indicador de éxito es la variedad de categorías CWE cubiertas y no la tasa de precisión sobre aplicaciones arbitrarias. Sin embargo, sí delimita el alcance de las afirmaciones posibles sobre la utilidad operativa del sistema en entornos de producción: un sistema que detecta correctamente el 100% de las vulnerabilidades de DVWA pero genera un volumen elevado de falsos positivos sobre aplicaciones reales tendría un valor operativo significativamente menor que el demostrado en el entorno controlado, ya que reintroduciría el problema de la fatiga de alertas que el componente de normalización busca mitigar. La estimación empírica de esta tasa requiere el estudio de caso múltiple sobre aplicaciones reales propuesto en la línea de trabajo futuro LT5, el cual constituye la condición necesaria para elevar la validez externa de la plataforma más allá del laboratorio. Hasta que dicho estudio sea realizado, la tasa de falsos positivos del sistema sobre aplicaciones reales permanece como una variable no observada, cuya magnitud es dependiente del stack tecnológico del objetivo, de la configuración de las herramientas y del nivel de agresividad de los escaneos.

8. CONCLUSIONES

El desarrollo y validación de esta plataforma de *Web Application Security Assessment* (WASA) ha demostrado que la automatización integral de la seguridad ofensiva no solo es viable, sino indispensable en los ciclos de desarrollo tecnológico actuales. A partir de los resultados obtenidos y la discusión planteada, se extraen las siguientes conclusiones fundamentales:

8.1 Viabilidad del Monitoreo Continuo (Dev SecOps)

Se comprobó empíricamente que el uso de un orquestador como n8n, configurado con nodos dinámicos de tiempo (Schedule) e iteración (Loop), permite trascender el enfoque tradicional de auditorías puntuales. El sistema logró ejecutar ciclos completos de descubrimiento y fuzzing sin intervención humana, garantizando la repetibilidad de las pruebas y reduciendo drásticamente las ventanas de exposición ante nuevas vulnerabilidades.

8.2 Sinergia y Eficiencia Multi-Motor

El ciclo del 24/05/2026 demostró con evidencia directa y verificable que la arquitectura de desacoplamiento asíncrono con Redis funciona tal como fue diseñada. El Worker de SQLMap confirmó una vulnerabilidad de Inyección SQL en `/vulnerabilities/brute/` de forma completamente autónoma, procesó 18 URLs en paralelo con el ciclo principal, manejó un timeout sin interrumpir el pipeline, y persistió sus resultados en PostgreSQL con anterioridad al cierre del ciclo de n8n. El total de 51 vulnerabilidades únicas registradas en BD — incluyendo 2 críticas de infraestructura Redis, 8 de alta severidad entre Nuclei, ZAP y

SQLMap Worker, y 8 de severidad media — constituye la evidencia empírica más completa producida por el sistema hasta la fecha.

8.3 Solución a la Fragmentación de Datos

El proceso de normalización implementado consolidó exitosamente 269 alertas brutas en 51 registros únicos abarcando 14+ categorías CWE distintas (Tabla 7), demostrando la viabilidad del enfoque de deduplicación multi-motor descrito en el Capítulo 4. Las limitaciones identificadas en el alcance actual del pipeline de ingesta —discutidas en la sección 7.4— y su hoja de ruta de validación exhaustiva se abordan en la línea de trabajo futuro LT7.

8.4 Transformación de Datos en Inteligencia Accionable

El Dashboard interactivo en React, con el KPI 'Total Vulnerabilidades' de 51 —tal como se observa en la Figura 5— y el ranking de endpoints revisado, demuestra mayor capacidad de priorización. Las 29 instancias de CSP no configurada (CWE-693) detectadas de forma transversal en la aplicación orientan los esfuerzos de remediación hacia la configuración del servidor Apache antes que hacia el análisis endpoint por endpoint, ya que corrigiendo la cabecera a nivel de servidor se resuelve la vulnerabilidad en todas las rutas afectadas simultáneamente.

8.5 Líneas de Trabajo Futuro

El presente trabajo ha demostrado la viabilidad técnica y operativa de un sistema de Web Application Security Assessment (WASA) automatizado y continuo. No obstante, como toda investigación aplicada enmarcada en el paradigma de Design Science Research (Hevner et al., 2004), el artefacto construido constituye una primera iteración de diseño cuya validez fue establecida sobre un entorno controlado y acotado. Los resultados reportados en el Capítulo 5, junto con las limitaciones reconocidas en la sección 7.8, permiten identificar con precisión las fronteras del sistema actual y, a partir de ellas, proyectar líneas de trabajo futuro que extienden y consolidan la contribución de esta tesis. Las siete líneas que se describen a continuación no son sugerencias genéricas: cada una responde a una tensión concreta detectada durante el desarrollo, y cada una abre un espacio de investigación con hipótesis propias y metodologías verificables.

LT1 — Integración nativa del sistema en pipelines de CI/CD con activación por evento de commit

La arquitectura implementada en esta tesis opera bajo un modelo de disparo temporal (Schedule Trigger), lo que implica que el análisis de seguridad se ejecuta en intervalos predefinidos con independencia de la actividad del repositorio de código fuente. Si bien este

modelo cumple con el principio de monitoreo continuo —y así fue validado en el criterio correspondiente de la Tabla 9—, no cierra la brecha entre el momento en que un desarrollador introduce un cambio en el código y el momento en que dicho cambio es evaluado por el sistema. En la configuración evaluada, esa brecha es de hasta 40 minutos; en despliegues con intervalos más espaciados puede extenderse a horas o días.

La línea de trabajo futuro propuesta consiste en reemplazar o complementar el nodo Schedule de n8n por un Webhook Trigger que reciba eventos nativos de plataformas de control de versiones (GitHub Actions, GitLab CI, Bitbucket Pipelines) en el momento exacto del push o del merge request. Bajo este esquema, cada commit que modifique rutas de la aplicación objetivo disparará automáticamente un ciclo de WASA focalizado sobre los endpoints afectados, reduciendo la ventana de exposición a minutos. Esta integración requeriría extender la arquitectura actual con un módulo de diff de superficie de ataque capaz de comparar el mapa de endpoints del ciclo anterior —almacenado en PostgreSQL— con el mapa resultante del nuevo Spider de ZAP, dirigiendo los recursos de escaneo hacia las rutas modificadas.

La viabilidad técnica de esta integración está documentada en la literatura: Putra y Kabetta (2022) demostraron mejoras cuantificables en la tasa de detección al integrar DAST en pipelines de CI/CD activados por evento, y Klooster et al. (2023) establecieron que sesiones de fuzzing cortas y frecuentes dentro de pipelines son efectivas y escalables. La hipótesis de investigación derivada es que la activación por commit, combinada con el análisis diferencial de superficie de ataque, elimina la ventana de exposición inherente al disparo temporal —de hasta 40 minutos en la configuración evaluada— al vincular el escaneo al evento que introduce el cambio en lugar de a un intervalo fijo, manteniendo la carga computacional por ciclo dentro de límites aceptables para entornos de desarrollo activos.

LT2 — Incorporación de análisis SAST como capa complementaria al pipeline DAST

El sistema desarrollado adopta un enfoque exclusivamente DAST (Dynamic Application Security Testing), analizando la aplicación en tiempo de ejecución sin acceso al código fuente. Esta decisión de diseño está fundamentada y justificada en el marco teórico (sección 2.3), pero implica una limitación estructural bien documentada: las herramientas DAST son incapaces de detectar vulnerabilidades que no se manifiestan en tiempo de ejecución bajo las condiciones de los payloads utilizados, tales como secretos expuestos en el código fuente, dependencias con CVEs conocidos que no generan respuestas HTTP observables,

o flujos de control inseguros que requieren un estado de sesión específico para ser alcanzados.

El estudio de Qadir et al. (2025), referenciado en el estado del arte, concluye que la adopción conjunta de SAST y DAST es ampliamente recomendada en la literatura pero raramente evaluada de forma comparativa y sistemática. Esta brecha identificada por los propios autores de esta tesis como parte del estado del arte representa una oportunidad directa de extensión: incorporar al pipeline orquestado por n8n un motor de análisis estático —como Semgrep, Bandit (para Python) o SonarQube Community Edition— que opere sobre el repositorio de código fuente en paralelo con los escaneos dinámicos.

La contribución técnica de esta línea residiría en el diseño de un módulo de correlación cruzada SAST-DAST capaz de unificar los hallazgos de ambas capas en la tabla de vulnerabilidades de PostgreSQL, asignando un campo de tipo de detección (static / dynamic / both) a cada registro. Un hallazgo confirmado por ambas capas recibiría mayor confianza que uno detectado por un único motor, reduciendo la tasa de falsos positivos —limitación reconocida en la sección 7.8 de esta tesis— sin necesidad de incrementar la agresividad de los escaneos dinámicos. La hipótesis a verificar sería que la correlación cruzada SAST-DAST reduce la tasa de falsos positivos del sistema integrado en un porcentaje estadísticamente significativo respecto a cualquiera de las dos capas operando en forma aislada.

LT3 — Escalabilidad horizontal del Worker asíncrono mediante un pool de consumidores Redis

La arquitectura de desacoplamiento asíncrono implementada en esta tesis —documentada en las secciones 4.2.4 y 4.3.1, y validada mediante el criterio de desacoplamiento de la Tabla 9— resuelve el problema de bloqueo del flujo principal de n8n al delegar los análisis de alta intensidad (SQLMap con nivel 4 y riesgo 2) a un Worker independiente en Python. Este diseño demostró su eficacia en el escenario experimental: un único Worker procesó la tarea encolada en Redis sin bloquear los escaneos concurrentes de Nuclei y ffuf.

Sin embargo, la arquitectura actual es inherentemente secuencial en lo que respecta al Worker: si n8n encola múltiples tareas de SQLMap simultáneamente —situación esperable cuando el Loop itera sobre una lista extensa de activos— el Worker único las procesa en orden FIFO, generando una cola de espera cuya longitud crece linealmente con el número de objetivos. En un escenario real con diez o más activos, el tiempo total de procesamiento

del Worker podría superar en horas el tiempo del ciclo principal de $n8n$, anulando parcialmente la ventaja del desacoplamiento.

La línea de trabajo futuro propone extender la arquitectura actual hacia un pool de Workers horizontalmente escalable, donde N instancias del script `worker_sqlmap.py` consuman concurrentemente la misma cola Redis mediante el patrón `Competing Consumers`. Este patrón, ampliamente documentado en la literatura de sistemas distribuidos, permite escalar la capacidad de procesamiento de forma elástica: en períodos de baja carga, una única instancia es suficiente; en períodos de alta demanda, el operador puede iniciar instancias adicionales sin modificar el código del orquestador ni del Worker. La gestión de instancias podría automatizarse mediante un supervisor de procesos como Supervisor o mediante orquestación en contenedores Docker Compose con la directiva `deploy.replicas`, extendiendo así el alcance del sistema hacia entornos de producción con múltiples activos bajo análisis simultáneo.

Complementariamente, esta línea podría explorar la incorporación de un sistema de priorización de tareas en la cola Redis —reemplazando la lista FIFO por una estructura de cola con prioridad (Sorted Set de Redis) — de modo que los activos de mayor criticidad (determinada por el historial de hallazgos almacenado en PostgreSQL) sean procesados antes que los de bajo riesgo histórico. Esta extensión convertiría al sistema en un escáner adaptativo, capaz de asignar recursos computacionales en función del riesgo acumulado de cada objetivo.

LT4 — Módulo de priorización de remediación basado en scoring contextual y deuda técnica acumulada

El Dashboard interactivo desarrollado en React (Capítulo 6) resuelve el problema de la fatiga de alertas al transformar el volumen de hallazgos brutos en visualizaciones ejecutivas accionables. No obstante, la priorización que ofrece actualmente se basa en dos métricas simples: la severidad individual de cada hallazgo (según escala CVSS) y la concentración de fallos por endpoint (Tabla 8). Si bien estas métricas son útiles, no capturan dimensiones del riesgo que son igualmente relevantes para la toma de decisiones en un equipo de desarrollo real.

En particular, el sistema actual no considera: (a) la persistencia temporal de un hallazgo, es decir, cuántos ciclos de escaneo consecutivos ha permanecido sin remediación; (b) la exposición pública del endpoint afectado, que determina si la vulnerabilidad es explotable

desde Internet o solo desde la red interna; (c) la presencia de exploits públicos disponibles para la vulnerabilidad (información disponible en las bases de datos NVD y ExploitDB a través de sus APIs públicas); ni (d) el contexto de negocio del endpoint, que determina si el dato que protege es crítico para la operación de la organización.

La línea de trabajo futuro propone diseñar e implementar un módulo de scoring contextual de remediación (Remediation Priority Score, RPS) que combine estas dimensiones en un índice compuesto. El RPS podría calcularse como una función ponderada de la severidad CVSS base, el número de ciclos sin remediar, un indicador binario de exposición pública y la disponibilidad de exploit público, con pesos ajustables por el operador según el perfil de riesgo de la organización. Este índice sería almacenado en una nueva columna de la tabla vulnerabilities de PostgreSQL y expuesto en el Dashboard como un criterio de ordenamiento adicional. La hipótesis a verificar mediante un estudio de caso con equipos de desarrollo reales es que la priorización por RPS reduce el tiempo medio de remediación de vulnerabilidades críticas en comparación con la priorización por severidad CVSS individual, al enfocarse en los hallazgos que combinan alta severidad con alta probabilidad de explotación activa.

Esta línea se conecta directamente con la brecha documentada en la sección 2.5.3 del estado del arte respecto a la visualización de métricas de seguridad como componente del proceso de evaluación, y extiende la contribución del Dashboard de una herramienta de visualización a una herramienta de gestión de riesgo con capacidad prescriptiva.

LT5 — Validación del sistema sobre aplicaciones reales en entornos de staging con autorización explícita

La totalidad del experimento documentado en esta tesis fue realizado sobre la aplicación Damn Vulnerable Web Application (DVWA) en un entorno de laboratorio aislado, de acuerdo con las consideraciones éticas y el alcance definidos en las secciones 1.5 y 3.8. Esta decisión metodológica es correcta y necesaria para la validación inicial del artefacto: un entorno con vulnerabilidades conocidas e intencionadas permite verificar que el sistema detecta lo que debe detectar sin ambigüedad. Sin embargo, esta misma condición introduce una limitación de validez externa (external validity) reconocida en la sección 3.6: los resultados no son directamente generalizables a aplicaciones reales, cuya superficie de ataque es más compleja, heterogénea y dinámica que la de DVWA.

La línea de trabajo futuro propone llevar a cabo un estudio de caso múltiple (multiple case study; Yin, 2018) en el que el sistema sea desplegado sobre aplicaciones reales en entornos de staging de organizaciones que otorguen su consentimiento explícito y por escrito para la realización de las pruebas. Un diseño adecuado seleccionaría al menos tres aplicaciones con perfiles tecnológicos distintos (por ejemplo: una aplicación PHP/MySQL, una aplicación Node.js/MongoDB y una aplicación Java/Spring Boot) para evaluar si la cobertura del sistema y la tasa de falsos positivos varían significativamente según el stack tecnológico del objetivo.

Este estudio permitiría responder preguntas de investigación que la presente tesis no puede responder sobre DVWA: ¿cuántos hallazgos reportados por el sistema son falsos positivos en aplicaciones reales? ¿Qué herramientas del conjunto (ZAP, Nuclei, ffuf, SQLMap) presentan mayor tasa de falsos negativos fuera del entorno controlado? ¿La normalización de datos implementada en el backend Node.js mantiene su efectividad ante formatos de salida no anticipados de las herramientas? Los resultados de este estudio de caso múltiple aportarían la evidencia empírica necesaria para elevar la validez externa de la proposición de diseño P1 más allá del entorno de laboratorio, acercando el sistema a condiciones de producción real con el rigor ético que la disciplina de seguridad ofensiva exige.

LT6 — Incorporación de un módulo de aprendizaje automático para reducción adaptativa de falsos positivos

El sistema actual depende de la precisión inherente de las herramientas de escaneo subyacentes para la calidad de sus hallazgos. Como se reconoce en la sección 7.8, al depender de motores de terceros como ZAP, Nuclei y SQLMap, el sistema hereda su tasa de falsos positivos, lo que requiere en última instancia la validación manual de un analista de seguridad. Esta dependencia del criterio humano para la confirmación de hallazgos es el principal obstáculo para que el sistema opere de forma completamente autónoma en un entorno de producción.

La línea de trabajo futuro propone incorporar un módulo de clasificación supervisada que aprenda, a partir de las decisiones históricas de los analistas humanos, a distinguir verdaderos positivos de falsos positivos sin intervención adicional. El mecanismo concreto consistiría en añadir al Dashboard un campo de retroalimentación por hallazgo (confirmed / false_positive / pending) que el analista complete durante el proceso de triage. Estos registros etiquetados, almacenados en PostgreSQL, constituirían el conjunto de entrenamiento de un clasificador binario —un modelo de gradient boosting como XGBoost o un clasificador de Random Forest— que tomaría como features la herramienta de origen

(source), el tipo de vulnerabilidad (type), el CWE ID, la evidencia (evidence) y el endpoint afectado (url).

A medida que el sistema acumule ciclos de escaneo y retroalimentación del analista, el clasificador aprendería los patrones de falsos positivos específicos del entorno objetivo, filtrando automáticamente los hallazgos de baja confianza antes de que lleguen al Dashboard. Este enfoque es coherente con la tendencia documentada en la literatura reciente de incorporar técnicas de aprendizaje automático en el análisis de alertas de seguridad para reducir la fatiga de alertas (Cheenepalli et al., 2025), y representa una extensión natural de la arquitectura de persistencia ya construida en esta tesis: la base de datos PostgreSQL, con su historial de hallazgos y su modelo de datos normalizado, provee el sustrato de datos necesario para entrenar y evaluar el modelo sin modificaciones estructurales significativas al sistema existente.

LT7 — Validación exhaustiva del pipeline de normalización

La verificación exhaustiva del pipeline de normalización, ejecutada sobre el JSON crudo del ciclo del 24/05/2026 y documentada en §5.2.2, confirmó la correspondencia exacta entre las 183 alertas brutas de ZAP y los 26 tipos únicos persistidos. La línea de trabajo futuro propone automatizar esa verificación como control permanente del pipeline, incorporando al workflow una etapa de reconciliación que compare el conteo bruto por severidad contra los registros persistidos en cada ciclo y emita una alerta ante cualquier divergencia, de modo que la completitud de la normalización deje de depender de una auditoría manual.

Reflexión de cierre

Las siete líneas de trabajo futuro descritas no son mutuamente excluyentes ni requieren ser abordadas en secuencia estricta. Las líneas LT1 y LT3 son extensiones de ingeniería de sistemas que pueden desarrollarse en paralelo con el sistema actual como base; LT2 es una extensión de cobertura que comparte la misma arquitectura de orquestación; LT4 es una extensión del componente de visualización que puede implementarse incrementalmente sobre el Dashboard existente; LT5 es un estudio empírico independiente que valida el sistema como un todo; y LT6 es una extensión de inteligencia que se vuelve viable una vez que el sistema haya acumulado suficiente historial de hallazgos etiquetados; LT7, por su

parte, es una tarea de consolidación técnica del pipeline actual, condición previa para que las restantes operen sobre datos completos. Juntas, estas líneas definen una agenda de investigación coherente que posiciona la plataforma desarrollada en esta tesis como la base de un programa de investigación más amplio en el campo del DevSecOps automatizado, contribuyendo a cerrar la brecha —documentada por Cheenepalli et al. (2025)— entre la disponibilidad de herramientas de análisis de seguridad y su integración continua, efectiva y accionable en los ciclos de vida reales del desarrollo de software.

9. REFERENCIAS BIBLIOGRÁFICAS

Abiola, O. B., & Olufemi, O. G. (2023). An enhanced CI/CD pipeline: A DevSecOps approach. *International Journal of Computer Applications*, 184(48), 8–13. <https://doi.org/10.5120/ijca2023922594>

Albahar, M., Alansari, D., & Jurcut, A. (2022). An empirical comparison of pen-testing tools for detecting web app vulnerabilities. *Electronics*, 11(19), 2991. <https://doi.org/10.3390/electronics11192991>

Altulaihan, E. A., Alismail, A., & Frikha, M. (2023). A survey on web application penetration testing. *Electronics*, 12(5), 1229. <https://doi.org/10.3390/electronics12051229>

Aydos, M., Aldan, Ç., Coşkun, E., & Soydan, A. (2022). Security testing of web applications: A systematic mapping of the literature. *Journal of King Saud University – Computer and Information Sciences*, 34(9), 6775–6792. <https://doi.org/10.1016/j.jksuci.2021.09.018>

Cheenepalli, J., Hastings, J. D., Ahmed, K. M., & Fenner, C. (2025). Advancing DevSecOps in SMEs: Challenges and best practices for secure CI/CD pipelines. *2025 IEEE 13th International Symposium on Digital Forensics and Security (ISDFS)*, 1–6. <https://doi.org/10.1109/ISDFS65363.2025.11011960>

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>

Klooster, T., Turkmen, F., Broenink, G., ten Hove, R., & Böhme, M. (2023). Continuous fuzzing: A study of the effectiveness and scalability of fuzzing in CI/CD pipelines. *Proceedings of the 2023 IEEE/ACM International Workshop on Search-Based and Fuzz Testing (SBFT)*, 25–32. <https://doi.org/10.1109/SBFT59156.2023.00015>

MITRE Corporation. (2025). *CWE – Common Weakness Enumeration*. <https://cwe.mitre.org/>

Nourry, O., Kashiwa, Y., Lin, B., Bavota, G., Lanza, M., & Kamei, Y. (2023). The human side of fuzzing: Challenges faced by developers during fuzzing activities. *ACM Transactions on*

Software Engineering and Methodology, 33(1), Art. 14, 1–26.
<https://doi.org/10.1145/3611668>

OWASP Foundation. (2021). *OWASP Top 10: The ten most critical web application security risks*. <https://owasp.org/www-project-top-ten/>

OWASP Foundation. (2025). *OWASP Zed Attack Proxy (ZAP) documentation*.
<https://www.zaproxy.org/docs/>

Prates, L., & Pereira, R. (2024). DevSecOps practices and tools. *International Journal of Information Security*, 24, Article 11. <https://doi.org/10.1007/s10207-024-00914-z>

Priyawati, D., Rokhmah, S., & Utomo, I. C. (2022). Website vulnerability testing and analysis of internet management information system using OWASP. *International Journal of Computer and Information System*, 3(3), 143–147.

ProjectDiscovery. (2025). *Nuclei: Fast and customizable vulnerability scanner based on simple YAML based DSL*. <https://github.com/projectdiscovery/nuclei>

Putra, A. M., & Kabetta, H. (2022). Implementation of DevSecOps by integrating static and dynamic security testing in CI/CD pipelines. *2022 IEEE International Conference on Computer Science and Information Technology (ICOSNIKOM)*, 1–6.
<https://doi.org/10.1109/ICOSNIKOM56551.2022.10034883>

Qadir, S., Waheed, E., Khanum, A., & Jehan, S. (2025). Comparative evaluation of approaches & tools for effective security testing of web applications. *PeerJ Computer Science*, 11, e2821. <https://doi.org/10.7717/peerj-cs.2821>

Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164.
<https://doi.org/10.1007/s10664-008-9102-8>

Singh, B. (2020). Automating security testing in CI/CD pipelines using DevSecOps tools: A comprehensive study. *SSRN Working Paper*. <https://doi.org/10.2139/ssrn.5267959>

Somi, V. (2023). Comparative analysis of DAST, SAST, and IAST: A comprehensive study on large-scale web applications. *Journal of Scientific and Engineering Research*, 10(8), 158–165. <https://doi.org/10.5281/zenodo.14005756>

Somi, V. (2024). A comparative analysis and benchmarking of dynamic application security testing (DAST) tools. *Journal of Engineering and Applied Sciences Technology*, 6(2).
[https://doi.org/10.47363/JEAST/2024\(6\)E139](https://doi.org/10.47363/JEAST/2024(6)E139)

Verizon. (2023). *2023 Data Breach Investigations Report (DBIR)*. Verizon Enterprise Solutions. <https://www.verizon.com/business/resources/reports/dbir/>

Wieringa, R. J. (2014). *Design science methodology for information systems and software engineering*. Springer. <https://doi.org/10.1007/978-3-662-43839-8>

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer.
<https://doi.org/10.1007/978-3-642-29044-2>

Yin, R. K. (2018). *Case study research and applications: Design and methods* (6.^a ed.). SAGE Publications.

10. ANEXOS

Anexo A: Comandos de Integración CLI (Fase de Escaneo y Fuzzing)

Ejemplos de las instrucciones ejecutadas localmente por los nodos *Execute Command* en n8n durante el ciclo de evaluación continua, inyectando dinámicamente el objetivo de análisis (TARGET).

1. Comando de ffuf (Descubrimiento de Directorios):

```
ffuf -u ${TARGET}/FUZZ -w ${WORDLIST} -mc 200,204,301,302,307,401,403 -s
```

```
-of json -o ${OUTPUT_FILE} -t 100 -timeout 15 -H "Cookie:
PHPSESSID=${PHPSESSID}; security=low"
```

2. Comando de SQLMap (Modo Automático/Batch):

```
python sqlmap.py -u
"http://localhost:8081/vulnerabilities/brute/?Login=Login&password=ZAP&u
sername=ZAP" \
--batch \
--random-agent \
--level=2 \
--risk=1 \
--cookie "PHPSESSID=e6ebtg50cn4a9mjd6f56r0duj2; security=low" \
--flush-session
```

3. Comando de Nuclei (Análisis DAST Avanzado basado en Templates):

```
nuclei -u ${TARGET} \
-t /Herramientas/nuclei-templates/ \
-severity low,medium,high,critical \
-json -o /Herramientas/nuclei_results.json
```

Anexo B: Estructura de Datos Normalizada (Output del Backend)

A continuación, se presenta un extracto del archivo estructurado servido por la API de Node.js/Express, el cual alimenta los gráficos y métricas del Dashboard en React. Se observa la correcta asignación de severidades y categorías OWASP/CWE:

```
{
  "scans": [
    { "id": 102, "date": "2026-03-24", "target":
"http://tienda-local.com" }
  ],
  "vulnerabilities": [
    {
      "id": 905,
      "scan_id": 102,
      "source": "ZAP",
      "type": "Inyección SQL (SQLi)",
      "severity": "High",
      "cvss_score": 8.5,
      "owasp_category": "A03:2021-Injection",
      "url": "http://tienda-local.com/product?id=1",
      "description": "El parámetro 'id' es vulnerable a inyección SQL
basada en errores.",
    }
  ]
}
```

```

    "cweid": 89,
    "evidence": "Error detectado: 'You have an error in your SQL
syntax'"
  },
  {
    "id": 906,
    "scan_id": 102,
    "source": "Nuclei",
    "type": "Directory Traversal",
    "severity": "High",
    "cvss_score": 7.5,
    "owasp_category": "A01:2021-Broken Access Control",
    "url":
"http://api.tienda-local.com/download?file=../../etc/passwd",
    "description": "Se logró leer archivos locales mediante Local File
Inclusion.",
    "cweid": 22,
    "evidence": "Payload: ../../../../etc/passwd"
  }
]
}

```

(La url <http://tienda-local.com> es solo un ejemplo ilustrativo del formato de salida y no un extracto del ciclo de validación documentado.)

Anexo C: Configuración del workflow n8n

El orquestador del sistema está implementado como un único workflow de n8n compuesto por 22 nodos, distribuidos en seis fases de ejecución. El archivo de configuración completo, [Flujo_Fuzzing_N8N.json](#), se publica íntegro en el repositorio de artefactos de este trabajo <https://github.com/Nickolan/Fuzzing-tesis>.

Versión publicada y su relación con el ciclo del 24/05/2026. El archivo corresponde al estado del workflow posterior a las correcciones documentadas en §7.4.1. No es un volcado del estado exacto del orquestador al momento de ejecutar el ciclo `scan_id=22`, que fue modificado con posterioridad.

El cambio funcional identificado es la corrección de la función `classifySeverity` (Anexo C.2.2), cuyo efecto sobre la clasificación de severidad está documentado en §7.4.1 y §5.3. Se registran además dos modificaciones que no alteran la salida del pipeline: la gestión de los parámetros de sesión, que pasó de estar declarada en cada nodo a centralizarse en el nodo inicial, y el orden de invocación de los motores. No es posible reconstruir con certeza el estado exacto del workflow al 24/05/2026, y por lo tanto no se afirma aquí una equivalencia literal entre ambos.

Lo que sí está verificado empíricamente es la equivalencia de salida: los ciclos `scan_id=105` y `scan_id=108` del 03/08/2026 (Tabla 10, Anexo G) fueron ejecutados con

esta versión del workflow sobre el mismo objetivo y bajo la misma configuración, y reprodujeron los 51 registros únicos y la distribución de severidad informados en la Tabla 6. El archivo publicado es, por lo tanto, el que un tercero debe utilizar para reproducir los resultados de este trabajo.

Su integridad puede verificarse mediante la suma SHA-256

`dd496d57473a5ce0e0d19b3b7c238a6b7f4847a9d62a116455cc82bd65add53d`. En el archivo publicado se sustituyeron por marcadores la clave de API de ZAP, el identificador de sesión de DVWA, la dirección de correo de notificación y los identificadores internos de credenciales de n8n; ninguna otra parte de la configuración fue modificada. El campo `name` del archivo conserva la denominación interna del workflow en n8n.

C.1 Inventario de nodos

#	Nodo	Tipo	Función
1	Schedule Trigger	scheduleTrigger	Dispara el ciclo en intervalos de 40 minutos
2	When clicking 'Execute workflow'	manualTrigger	Disparo manual alternativo, para pruebas
3	URL Ejemplo	code	Define los objetivos y los parámetros de sesión; marca el inicio del ciclo
4	Loop Over Items	splitInBatches	Itera sobre los objetivos de a uno (<code>batchSize: 1</code>)
5	Crear ID	postgres	INSERT en <code>scans</code> ; genera el <code>scan_id</code> del ciclo
6	ZAP Spider (Descubrimiento)	code	Crawling autenticado; devuelve las URLs descubiertas
7	ffuf	code	Fuzzing de directorios sobre el objetivo
8	Alertas ZAP	code	Escaneo activo y recuperación de alertas vía API
9	Nuclei Scann	code	Escaneo por plantillas sobre aplicación e infraestructura
10	Item Lists	splitOut	Expande la lista de

			URLs descubiertas
11	Filter	filter	Retiene las URLs con parámetros inyectables
12	Redis	redis (push)	Encola las tareas en <code>sqlmap_tasks</code> para el Worker
13	Combinación de Datos	merge	Une las salidas de ffuf, ZAP y Nuclei
14	Consolidar Resultados Nuclei	code	Normaliza la severidad y deduplica por <code>type</code>
15	Reporte Final	code	Calcula métricas agregadas del ciclo
16	Edit Fields	set	Propaga el <code>scan_id</code> a la etapa de persistencia
17	Insertar Escaneos	postgres	Actualiza el registro del ciclo en <code>scans</code>
18	Preparar las vulnerabilidades	code	Mapea los hallazgos al esquema de <code>vulnerabilities</code>
19	Insertar Vulnerabilidades	postgres	INSERT de los registros normalizados
20	reporteHTML	markdown	Convierte el reporte a HTML
21	AddHTML	code	Compone el cuerpo del correo
22	Send email	emailSend	Emite la notificación y realimenta el Loop

C.2 Fragmentos que respaldan afirmaciones del cuerpo

Se reproducen a continuación los cuatro fragmentos del workflow que sostienen afirmaciones específicas de los Capítulos 4, 5 y 7.

C.2.1 — Clave de deduplicación (nodo `Consolidar Resultados Nuclei`). Respalda la declaración de §5.4 y la nota metodológica de la Tabla 8: la clave es el campo `type` y no preserva la URL.

```
function addOrUpdateVulnerability(vuln) {
  const key = `${vuln.type}`;
  if (!vulnerabilityMap.has(key)) {
```



```
vulnerabilityMap.set(key, vuln);
}
}
```

C.2.2 — Normalización de severidad (nodo **Consolidar Resultados Nuclei**).

Respalda §7.4.1: la función corregida no promueve **High** a **Critical**, y mapea los niveles informativos al nivel Bajo de la taxonomía unificada.

```
function classifySeverity(risk) {
  if (!risk) return 'low';
  const riskMap = {
    'High': 'high', 'Medium': 'medium', 'Low': 'low',
    'Informational': 'low',
    'critical': 'critical', 'high': 'high',
    'medium': 'medium', 'low': 'low', 'info': 'low'
  };
  return riskMap[risk] || risk || 'low';
}
```

C.2.3 — Encolamiento asíncrono (nodo **Redis).** Respalda el diagrama de secuencia de la Figura 2 y el análisis de §7.3.

```
operation:  push
list:      sqlmap_tasks
messageData: {{ JSON.stringify({
  "urls": $json.urlsFound,
  "scan_id": $('Crear ID').item.json.id,
  "cookie": "PHPSESSID=<PHPSESSID>; security=low",
  "level": 2, "risk": 1 }) }}
tail:      true
```

C.2.4 — Iteración sobre objetivos (nodo **Loop Over Items).** Respalda la afirmación de §4.2 sobre el patrón de iteración dinámica sobre múltiples activos.

```
{ "batchSize": 1, "options": { "reset": false } }
```

Anexo D: Lógica de Notificación (Markdown a HTML)

El flujo finaliza con la transformación estética del reporte. El nodo "reporteHTML" convierte la estructura **.md** generada localmente a HTML. Posteriormente, un nodo "Code in JavaScript" inyecta los estilos CSS antes de pasarlo al nodo final "Send email",

garantizando que el formato enviado sea idéntico al reporte local generado en las etapas tempranas de consolidación.

Anexo E: Estructura del Modelo Relacional en PostgreSQL

Script DDL utilizado para la creación de la base de datos `db_fuzzing` y sus tablas asociadas, permitiendo almacenar la información recopilada por el nodo "Insertar Escaneos" y "Insertar Vulnerabilidades".

```
CREATE DATABASE db_fuzzing;
\c db_fuzzing;
```

```
CREATE TABLE scans (

    id SERIAL PRIMARY KEY,
    target_url VARCHAR(500) NOT NULL,
    scan_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_vulnerabilities INTEGER DEFAULT 0,
    critical_count INTEGER DEFAULT 0,
    high_count INTEGER DEFAULT 0,
    medium_count INTEGER DEFAULT 0,
    low_count INTEGER DEFAULT 0,
    report_path VARCHAR(500)
);
```

```
CREATE TABLE vulnerabilities (
    id SERIAL PRIMARY KEY,
    scan_id INTEGER REFERENCES scans(id) ON DELETE CASCADE,
    source VARCHAR(50),
    type VARCHAR(200),
    severity VARCHAR(20),
    url TEXT,
    description TEXT,
    solution TEXT,
    cweid INTEGER,
    evidence TEXT
);
```

Nota: el esquema no incluye una columna `cvss_score` porque la versión actual del pipeline no calcula un puntaje CVSS normalizado por hallazgo; la severidad se determina mediante la función `classifySeverity` a partir de la escala nativa de cada herramienta (sección 7.4.1). El campo `cvss_score` del Anexo B es ilustrativo del formato de salida deseado para una versión futura del sistema, no del estado actual.

Anexo F: Script del Worker de Seguridad Asíncrono

Este código representa el "músculo" operativo que reside fuera del orquestador n8n para permitir escaneos de alta intensidad.

```
import redis
import subprocess
import json
import psycpg2
import re
import logging

# --- CONFIGURACIÓN DE LOGS ---
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger("sqlmap-worker")

# --- CONFIGURACIÓN ---
REDIS_CONF = {'host': 'localhost', 'port': 6379, 'db': 0}
DB_CONF = {
    'dbname': 'db_fuzzing',
    'user': 'TU_USUARIO',
    'password': 'TU_CONTRASEÑA',
    'host': 'localhost'
}

r = redis.Redis(**REDIS_CONF)

def extract_payload(stdout_text):
    title_match = re.search(r'Title:\s*(.+)', stdout_text)
    payload_match = re.search(r'Payload:\s*(.+)', stdout_text)

    if title_match and payload_match:
        return title_match.group(1).strip(), payload_match.group(1).strip()
    return None, None

def save_to_postgres(url, title, payload, scan_id):
    try:
        logger.info(f"📁 Intentando guardar hallazgo en BD para {url}...")
        conn = psycpg2.connect(**DB_CONF)
        cur = conn.cursor()
```

```

        query = """
            INSERT INTO vulnerabilities (scan_id, source, type, cweid,
            severity, url, description, solution, evidence)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
        """
        evidence = f"Técnica: {title}\nPayload: {payload}"

        # [FIX 2026-07-27] Severidad hardcodeda corregida de 'critical' a
        'high':
        # una inyección SQL confirmada corresponde a CWE-89 / Alto según la
        # clasificación usada en el resto de la tesis (Tabla 7), no a
        Crítico.
        # Consistente con el mismo fix aplicado en el nodo n8n
        # "Consolidar Resultados Nuclei" (sección 3, procesamiento de
        SQLMap).
        cur.execute(query, (
            scan_id,
            'SQLMap (Worker)',
            'SQL Injection',
            89,
            'high',
            url,
            'Vulnerabilidad detectada mediante escaneo asíncrono profundo.',
            'Implementar consultas parametrizadas.',
            evidence
        ))
        conn.commit()
        cur.close()
        conn.close()
        logger.info(f"✅ Hallazgo guardado exitosamente en BD para el
        scan_id={scan_id}")
    except Exception as e:
        logger.error(f"❌ Error al guardar en Postgres: {e}")

    logger.info("🛡️ Worker de Seguridad Activo. Esperando tareas...")

    while True:
        _, task_raw = r.blpop('sqlmap_tasks', 0)

        try:
            task = json.loads(task_raw)

```

```
except json.JSONDecodeError:
    logger.warning(f"⚠ Basura o mensaje inválido ignorado en Redis: {task_raw}")
    continue

# Agregamos un log para ver exactamente qué JSON está llegando
logger.info(f"📦 Nueva tarea recibida: {task}")

# ¡CORRECCIÓN CRÍTICA AQUÍ! Agregamos 'urlsFound' que es lo que manda
n8n
targets_url = task.get('urlsFound') or task.get('urls') or
task.get('url') or []
current_scan_id = task.get('scan_id', 'Desconocido')

if isinstance(targets_url, str):
    targets_url = [targets_url]

logger.info(f"🔍 URLs a procesar encontradas: {len(targets_url)}")

for target_url in targets_url:
    if not target_url:
        continue

    logger.info(f"🚀 Iniciando SQLMap en: {target_url}")

    level = task.get('level', 2)
    risk = task.get('risk', 1)
    cookie = task.get('cookie', "security=low")

    newCommand = [
        "python",
        "C:\\Users\\Nicolas\\Herramientas\\sqlmap\\sqlmap.py",
        "-u", target_url,
        "--batch",
        "--random-agent",
        f"--level={level}",
        f"--risk={risk}",
        "--cookie", cookie,
        "--flush-session"
    ]
```

```
logger.info(f"⚙️ Comando a ejecutar: {newCommand}")

try:
    logger.info("⌚ Ejecutando Subprocess de SQLMap... (esto puede tardar varios minutos)")
    result = subprocess.run(newCommand, capture_output=True, text=True, timeout=600)
    logger.info("✅ Subprocess de SQLMap finalizado.")

    print("🔍 Analizando resultados de SQLMap...")

    stdout_lower = result.stdout.lower()
    es_vulnerable = (
        "is vulnerable" in stdout_lower or
        "injection point(s)" in stdout_lower or
        ("parameter:" in stdout_lower and "payload:" in stdout_lower)
    )
    #logger.info(f"🇮🇹 Código de salida: {stdout_lower}")

    if es_vulnerable:
        logger.info(f"🔥 ¡VULNERABILIDAD ENCONTRADA en {target_url}!")

        try:
            with
open("C:\\Users\\Nicolas\\Herramientas\\reporte_vulnerabilidad.txt", "w", encoding="utf-8") as f:
                f.write(result.stdout)
            except Exception as e:
                logger.error(f"⚠️ No se pudo guardar el txt: {e}")

        title, payload = extract_payload(result.stdout)

        # Modificamos esta validación para evitar falsos negativos al guardar en la BD
        if title or payload:
            # Usamos 'Desconocido' si no pudo parsear alguno, para no perder la alerta
            save_to_postgres(target_url, title or "Múltiples/No
```

```
parseado", payload or "Revisar TXT", current_scan_id)
    else:
        logger.warning("⚠ Hay vulnerabilidad, pero el regex no
        extrajo el payload. Guardando alerta básica.")
        save_to_postgres(target_url, "SQLi Confirmado", "Revisar
        logs/TXT de SQLMap", current_scan_id)

    else:
        logger.info(f"✅ Escaneo limpio para {target_url}. No se
        encontraron vulnerabilidades.")

    except subprocess.TimeoutExpired:
        logger.error(f"⌚ Timeout al escanear {target_url}. La tarea
        alcanzó los 10 minutos y fue cancelada.")
```

Nota de corrección: la versión inicial del Worker (disponible en el historial de commits) construía el comando de SQLMap mediante interpolación de strings, lo que representaba un riesgo de inyección de comandos si la URL encolada no era de confianza. La versión final, incluida en este anexo, construye el comando como una lista de argumentos independientes, eliminando ese vector.

Se corrigió además el mensaje de log del manejador de `TimeoutExpired`, que declaraba 5 minutos cuando el parámetro `timeout` configurado en `subprocess.run` es de 600 segundos (10 minutos).

Anexo G: Evidencia de Verificación Post-Corrección

Este anexo documenta capturas del Dashboard correspondientes a los dos ciclos de verificación independientes ejecutados tras la corrección del defecto de mapeo de severidad descrito en la sección 7.4.1, presentados como evidencia visual complementaria a la Tabla 10.

Las capturas incorporan además una corrección menor al KPI "Escaneos Realizados", el cual originalmente mostraba el conteo histórico total de escaneos en la base de datos independientemente del filtro de `scan_id` aplicado; la tarjeta fue renombrada a 'Escaneo Analizado', mostrando el identificador del `scan_id` filtrado.

Las figuras de este anexo se numeran como Figura G.1 y G.2, esquema independiente de la numeración correlativa Figura 1 a Figura 9 utilizada en el cuerpo del documento.

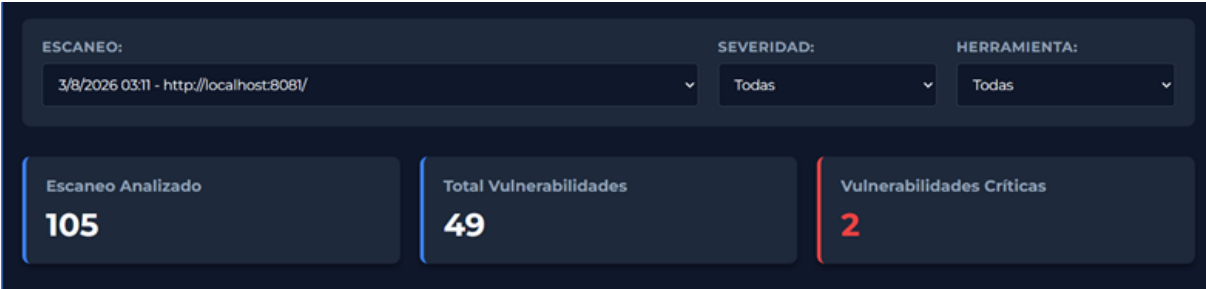


Figura G.1. Cabecera del Dashboard — scan_id=105 (03/08/2026, 00:11 hs). KPI de vulnerabilidades críticas correctamente reflejado en 2, consistente con los dos hallazgos de infraestructura Redis (CVE-2025-49844 y CVE-2025-46817).

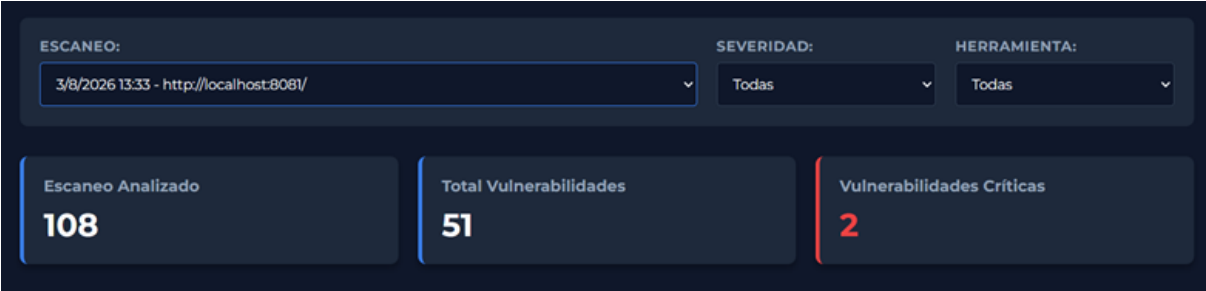


Figura G.2. Cabecera del Dashboard — scan_id=108 (03/08/2026, 10:33 hs). Segundo ciclo de verificación independiente, reproduciendo el mismo conteo de vulnerabilidades críticas (2) obtenido en el ciclo anterior.

